

[MORNING – ATLAS]: A FAST AND LIGHTWEIGHT DIGITAL SIGNATURE FROM MODULE LEARNING WITH ROUNDING

Supriya Adhikary, Debayan Das, Abhinava De, Anindya Ganguly, Harry
Hart, Angshuman Karmakar, Suparna Kundu, Chaoyun Li, Prajna Mahadev,
Puja Mondal, Quinten Norga, Debranjana Pal, Karthick Srivatsan

[ATLAS]

Email: pqatlas@gmail.com

Website: <https://sites.google.com/view/morningatlas>

June 30, 2026

Contents

NOTATIONS AND SYMBOLS	6
ABBREVIATIONS	7
1 INTRODUCTION	8
1.1 Features of our scheme	8
1.2 Novelty of ATLAS	9
1.3 Basic Cryptographic Primitives	10
1.3.1 Identification Algorithm	10
1.3.2 Fiat-Shamir Signatures	10
2 ATLAS DESIGN DESCRIPTIONS	11
2.1 Design Rationale	11
2.2 ATLAS Specification	12
2.3 Supporting Algorithms	13
2.4 Correctness of the Signature Scheme	16
2.5 Analysis of Failure Probability: Rejection Sampling	17
2.6 Criteria for Parameter Selection	20
2.7 Communication Cost: Key and Signature Size	22
2.7.1 Key Sizes	22
2.7.2 Signature Size	23
3 SECURITY ARGUMENTS	23
3.1 Hard Problems	23
3.2 Formal Security	24
3.3 Algebraic Security	26
3.3.1 Calculating the practical security for ATLAS	28
3.4 Security Beyond Unforgeability	29
3.5 Side-Channel Security	29

4	IMPLEMENTATION OF ATLAS	30
4.1	Implementation Parameters	30
4.2	Algebraic Structures and Ring Arithmetic	31
4.2.1	Data structure.	31
4.2.2	Arithmetic in $\mathcal{R}_q$	31
4.3	Polynomial Multiplication	32
4.3.1	Striding Toom-Cook and Karatsuba Multiplication	33
4.3.2	Design Space Exploration and Multiplication Strategies	36
4.4	Packing and Unpacking	38
4.4.1	Packing and Unpacking of Vectors	38
4.4.2	Algorithms for Packing and Unpacking: Keys and Signature	39
4.5	Symmetric Primitives	40
4.5.1	Hashing to the Challenge Set	40
4.6	Sampling Randomness	41
4.6.1	Generation of the Public Matrix	41
4.6.2	Sampling Challenges.	41
4.7	NGCC API Implementation	41
4.8	Known Answer Test Values	42
5	PERFORMANCE EVALUATION	42
5.1	Performance Test	42
5.1.1	Performance Comparison of Polynomial Multiplication Strategies	42
5.1.2	Reference and Optimized Implementation of ATLAS	44
5.2	Performance Analysis, Considerations and Trade-offs	45
5.3	Hardware Implementation	46
6	CONCLUSION	46
6.1	Advantages of ATLAS	46
6.2	Constraints and Tradeoffs	47
6.3	Additional Security Level: 384-bit.	47

7	BIBLIOGRAPHY	48
A	MISSING PROOFS	50
A.1	Min-Entropy	50
A.2	Proof of EUF-KOA Security	52
B	MISSING ALGORITHMS	54
B.1	Polynomial Multiplication	54
C	SPECIFICATION SUBROUTINES	57
D	AUTHOR DETAILS	58

Abstract

This document presents the technical specification for **ATLAS**, a high-performance digital signature scheme designed for the post-quantum era. Developed in alignment with the standards established by the Institute of Commercial Cryptography Standards (ICCS), this scheme adheres to the guidelines of the Next-generation Commercial Cryptographic Algorithms Program (NGCC) to provide robust, scalable, and efficient digital authentication.

The design of **ATLAS** follows the *Fiat–Shamir with Aborts* paradigm and derives its security from the hardness of the Module Learning with Rounding (MLWR) problem. Throughout the construction, we employ the standard rounding operation $\lfloor x \rfloor := \lfloor x + \frac{1}{2} \rfloor$. The use of deterministic rounding eliminates the need to store and process additional noise polynomials, leading to reduced secret-key storage and lower computational overhead. At the same time, the overall structure remains closely aligned with that of Dilithium, allowing us to leverage its well-studied design principles and the extensive cryptanalytic understanding developed for modern lattice-based signatures.

Following the NGCC recommendations, we provide parameter sets for the 128, 256, and 512-bit classical security levels λ , which correspond to approximately 64, 128, and 256 bits of quantum security Λ , respectively. Additionally, this specification includes a dedicated 192-bit classical (approx. 96-bit quantum) security level, offering a complete implementation suite for a wide range of commercial applications. For the 128-bit security level of **ATLAS**, the public key, secret key, and signature sizes are 1328, 2128, and 2081 bytes, respectively. The concrete storage and communication requirements for each security tier are detailed in Table 1, highlighting the scheme’s optimized footprint across the proposed parameters.

The optimized implementation accelerates performance through AVX2-vectorized arithmetic routines and a strided hierarchical polynomial multiplication strategy based on Toom–Cook and Karatsuba decomposition, where negacyclic reduction modulo $x^n + 1$ is handled implicitly throughout the recursion. The decomposition configurations yielding the best practical performance are two levels of Toom-4 followed by degree-8 schoolbook multiplication for $n = 128$; two levels of Toom-4, one level of Karatsuba, and degree-8 schoolbook multiplication for $n = 256$; and three levels of Toom-4 followed by degree-8 schoolbook multiplication for $n = 512$.

Table 1: Cryptographic Sizes (in bytes) by Security Level

Classical Security (λ)	128-bit	192-bit	256-bit	512-bit
Quantum Security(Λ)	80-bit	96-bit	128-bit	256-bit
Metric	ATLAS-128	ATLAS-192	ATLAS-256	ATLAS-512
Public Key Size	1328	2112	2848	6688
Secret Key Size	2128	3152	4016	7920
Signature Size	2081	3365	4656	10081

NOTATIONS AND SYMBOLS

Notation / Symbol	Meaning
$\lfloor v \rfloor := \lfloor v + \frac{1}{2} \rfloor$	Standard rounding function.
λ (Λ)	Classical (quantum) security level.
n	Degree of the cyclotomic polynomial defining the ring dimension.
q, p	Primary and rounded moduli, respectively.
k, l	Row and column dimensions of the public matrix.
$\mathbb{Z}_q$ ($\mathbb{Z}_p$)	Integers modulo q (p).
$\mathcal{R}$	Polynomial ring $\mathbb{Z}[X]/\langle X^n + 1 \rangle$.
$\mathcal{R}_q$	Quotient ring $\mathbb{Z}_q[X]/\langle X^n + 1 \rangle$.
$\mathcal{R}_q^m$	Rank- m module over $\mathcal{R}_q$.
$\mathcal{R}_p$	Rounded target ring modulo p .
$\mathbf{A}$	Public matrix in $\mathcal{R}_q^{k \times l}$.
ρ	Seed used to deterministically expand the public matrix.
$\mathbf{s}_1$	Secret short vector used in key generation and signing.
$\mathbf{s}_2$	Rounding-induced auxiliary secret/error vector.
$\mathbf{t}, \mathbf{t}_1, \mathbf{t}_0$	Public key vector and its high-/low-order decomposed parts.
$\mathbf{y}$	Ephemeral masking vector sampled during signing.
$\mathbf{w}, \mathbf{w}_1$	Commitment vector and its high-order decomposed part.
ξ_1, ξ_2, ν	Intermediate rounding and compensation terms used during signing.
c	Sparse challenge polynomial/output of the Fiat–Shamir transform.
μ	Message digest used to derive the challenge.
$\mathbf{z}$	Signature response vector.
$\mathbf{h}$	Hint vector used by the verifier to recover high-order bits.
σ	Signature, typically written as $(\mathbf{z}, \mathbf{h}, c)$.
pk, sk	Public key and secret key, respectively.
κ	Challenge weight parameter.
γ_1, γ_2	Masking and decomposition bounds used in signing.
$\bar{\gamma}_2$	Scaled decomposition/correctness bound after rounding to modulus p .
β_1, β_2	Rejection-sampling bounds used for $\mathbf{z}$ and low-order terms.
η	Noise parameter.
ω	Maximum weight of the hint vector.
ζ, ζ'	SVP-related bounds used in the security analysis.
$\ \cdot\ $	Norm operator used for vector and polynomial bounds.
$\ \cdot\ _\infty$	Infinity norm.
bits_δ	bit-size of δ
CRH	Collision resistant hash function, with output of 48-bytes.
$\mathcal{B}(\text{mean}, \text{var})$	Binomial distribution with mean mean , and variance var
$\mathcal{U}(\text{mean}, \text{var})$	Uniform distribution with mean mean , and variance var
$\mathcal{N}(\text{mean}, \text{var})$	Normal distribution with mean mean , and variance var

ABBREVIATIONS

Abbreviation	Meaning
LWE	Learning With Errors.
LWR	Learning With Rounding.
MLWE	Module Learning With Errors.
MLWR	Module Learning With Rounding.
SIS	Short Integer Solution.
MSIS	Module Short Integer Solution.
SelfTargetMSIS	Self-Target Module Short Integer Solution.
EUF-CMA	Existential Unforgeability under Chosen-Message Attack.
SUF-CMA	Strong Existential Unforgeability under Chosen-Message Attack.
EUF-KOA	Existential Unforgeability under Key-Only Attack.
HVZK	Honest-Verifier Zero-Knowledge.
NTT	Number Theoretic Transform.
XOF	eXtendable-Output Function.
CRH	Collision-Resistant Hash.
SVP	Shortest Vector Problem.
NGCC	National Guidelines for Commercial Cryptography.
ICCS	Institute of Commercial Cryptography Standards.
KAT	Known Answer Test.
KDF-SM3	Key-derivation-function construction based on SM3.
DSA	Digital Signature Algorithm.
KEM	Key Encapsulation Mechanism.
CDF	Cumulative Distribution Function.
PDF	Probability Distribution Function.

1 INTRODUCTION

Lattice-based cryptography is a leading candidate for security against quantum computers. Its efficiency and performance are demonstrated by the recent standardisation of CRYSTALS-Dilithium [13] by the National Institute of Standards and Technology [2] and HAETAE [10] in South Korea [17].

We introduce **ATLAS**, a digital signature scheme using the Fiat-Shamir with aborts framework. While Okada *et al.* [23] proposed a similar approach, our design rationale is different. Our construction provides parameters for 128-bit, 192-bit, 256-bit, and 512-bit security levels.

Although **ATLAS** follows design choices from Dilithium, its security is based on the Learning with Rounding (LWR) problem. This problem was first proposed by Banerjee *et al.* [4], and it is a variant of the well-known Learning with Errors (LWE) [25]. In LWR problem, deterministic rounding replaces the random noise. This approach makes **ATLAS** a fast, lightweight, and compact signature scheme for resource-constrained devices.

1.1 Features of our scheme

The design of the **ATLAS** signature scheme prioritizes a rigorous balance between cryptographic hardness, computational efficiency, and both storage and transmission costs. We present **ATLAS** as a digital signature primitive whose security is fundamentally rooted in the computational hardness of finding short vectors in module lattices. The scheme was engineered with the following primary design criteria in mind.

- **Innovativeness.** **ATLAS** is engineered for high parallelism and a reduced memory footprint. The specific parameter selection, such as the varied ring degrees from $n = 128$ to $n = 512$, enables fine-grained control over the security-performance tradeoff. This makes the algorithm highly innovative for both modern software environments and specialized hardware accelerators.
- **Simplicity.** The **ATLAS** construction is deliberately simple, relying on the well-understood Module Learning with Rounding (MLWR) hard problem. As shown in Table 2, it utilizes power-of-two moduli (such as $q = 2^{23}$ and $p = 2^{18}$), which simplifies the arithmetic by replacing complex modular reductions with efficient bitwise operations and eliminating the need for rejection sampling.
- **Flexibility.** The scheme is designed to be platform-agnostic. The primary arithmetic operations, particularly polynomial multiplication, can be optimized using various techniques depending on the target architecture. The flexible choice of dimensions (k, l) and ring degrees (n) across the security levels allows the scheme to be tailored for either ultra-low-latency or high-throughput requirements.
- **Compatibility.** **ATLAS** is designed to be part of a broader suite of next-generation post-quantum cryptographic primitives. Based on the MLWR hard problem, it maintains structural compatibility with other MLWR-based designs, enabling integrated systems to share underlying hardware modules or software libraries for modular arithmetic, thereby reducing the overall implementation surface.
- **Performance Advantage.** **ATLAS** offers a distinct advantage in terms of communication overhead and memory usage. As detailed in Table 1, the 128-bit security level achieves a public key size of only 1328 bytes. Furthermore, the use of power-of-two arithmetic ensures high efficiency in

embedded devices and facilitates side-channel protection without the expensive overhead of masked multiplication.

- **Extensibility.** The architectural framework of **ATLAS** is engineered for cross-primitive utility beyond digital signatures. This software IP can be readily repurposed to implement **MLWR** variants or Key Encapsulation Mechanisms (KEMs) with minimal logic overhead. The granular scaling of ring degrees ($n = 128, 256, 512$) allows the scheme to be extended into advanced cryptographic protocols, such as Aggregated Signatures and Multi-party Threshold Schemes, where smaller polynomial operations are required in parallel.

1.2 Novelty of ATLAS

The **ATLAS** signature scheme follows the Fiat-Shamir with aborts paradigm shared by **Dilithium** [13] and **HAETAE** [10], but differs in two concrete ways. First, unlike **Dilithium**, which generates its public key via the **MLWE** relation $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ requiring explicit noise sampling, and unlike **HAETAE**, which retains **MLWE** with a bimodal Gaussian secret distribution, **ATLAS** adopts an **MLWR**-based public key $\mathbf{t} = \lfloor \frac{p}{q} \mathbf{A}\mathbf{s}_1 \rfloor$, where rounding replaces noise sampling entirely. This eliminates the error term $\mathbf{s}_2$ and avoids discrete Gaussian sampling, reducing both key generation cost and secret key size. Second, while both **Dilithium** and **HAETAE** use **SHAKE-256** as their symmetric primitive, **ATLAS** instantiates all symmetric operations via the **KDF-SM3** framework [27], making it suitable for deployment in environments where **SM3** is the mandated cryptographic standard.

Differences Between ATLAS and MLWRSign. **ATLAS** introduces several design improvements over **MLWRSign** [23], both in parameter selection and implementation. Unlike **MLWRSign**, which fixes the ring degree to $n = 256$ for all security levels, **ATLAS** adopts security-level-specific ring dimensions and selects the smallest feasible ring that satisfies the required security. Concretely, **ATLAS** uses $n = 128$ for **ATLAS-128** and **ATLAS-192**, $n = 256$ for **ATLAS-256**, and $n = 512$ for **ATLAS-512**. Similarly, while **MLWRSign** fixes the challenge weight to $\kappa = 60$ across all parameter sets, **ATLAS** selects κ according to the target security level while ensuring that the challenge space exceeds the desired security level. Consequently, **ATLAS** employs $\kappa = 31$ for **ATLAS-128** and $\kappa = 60$ for **ATLAS-256** and higher security levels. These design choices also affect the remaining scheme parameters, including module dimensions, rejection-sampling bounds, and compression parameters, resulting in parameter sets optimized for each security level. Additionally, **ATLAS** provides parameters for a higher security level, 512-bit security.

The second major difference lies in the implementation strategy. Since **ATLAS** employs a power-of-two modulus q , conventional NTT-based polynomial multiplication is not applicable. Instead, polynomial arithmetic is built upon a hierarchical decomposition framework combining Toom–Cook and Karatsuba algorithms, with degree-8 schoolbook multiplication serving as the base case. The complete design is presented in Section 4.3. Empirical evaluation shows that the following decomposition strategies provide the best performance:

- $n = 128$: two levels of Toom-4 followed by degree-8 schoolbook multiplication;
- $n = 256$: two levels of Toom-4, one level of Karatsuba, followed by degree-8 schoolbook multiplication;
- $n = 512$: three levels of Toom-4 followed by degree-8 schoolbook multiplication.

All polynomial arithmetic routines are implemented specifically for the selected parameter sets rather than using generic implementations. We further evaluate strided variants of these decomposition strategies and observe consistent improvements in both execution time and memory utilization over their non-strided counterparts.

1.3 Basic Cryptographic Primitives

1.3.1 Identification Algorithm

A canonical *identification scheme* IDS is a three-pass interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ defined over a challenge set $\mathcal{C}$. The exchange consists of a commitment W , a random challenge $c \leftarrow \mathcal{C}$, and a response R , culminating in a deterministic decision by $\mathcal{V}$ based on the transcript $\pi = (W, c, R)$ and the public key pk .

Definition 1.1 (Canonical Identification Scheme). *A canonical identification scheme $\text{IDS} = (\text{IDS.Gen}, \text{IDS.P}, \text{IDS.V})$ on a challenge set $\mathcal{C}$ consists of three polynomial-time algorithms:*

- $\text{IDS.Gen}(1^\lambda) \rightarrow (pk, sk)$: A probabilistic algorithm that outputs a public key pk and a secret key sk .
- $\text{IDS.P}(sk) \rightarrow \pi$: An interactive algorithm that takes sk as input, and outputs a proof transcript π .
- $\text{IDS.V}(pk, \pi) \rightarrow 0/1$: An algorithm that takes pk , and π as input, and outputs 1 if it verifies, else 0.

The interactive protocol IDS.P between the prover $\mathcal{P}$ and the verifier $\mathcal{V}$ proceeds in three passes as follows:

1. **Commitment:** The prover $\mathcal{P}$ computes a commitment W (using internal randomness r) and sends it to the verifier $\mathcal{V}$.
2. **Challenge:** $\mathcal{V}$ samples a challenge c uniformly at random from $\mathcal{C}$ (i.e., $c \leftarrow \mathcal{C}$) and sends it to $\mathcal{P}$.
3. **Response:** $\mathcal{P}$ computes a response R based on (sk, r, c) and sends it to $\mathcal{V}$.

Finally, $\mathcal{P}$ outputs $\pi = (W, c, R)$ as the proof transcript and sends it to the verifier. The verifier $\mathcal{V}$ runs IDS.V , and outputs a bit $b \in \{0, 1\}$.

1.3.2 Fiat-Shamir Signatures

Let $\text{IDS} = (\text{IDS.Gen}, \text{IDS.P}, \text{IDS.V})$ be a canonical identification scheme, κ_m be a positive integer, and $\mathcal{H} : \{0, 1\}^* \rightarrow \mathcal{C}$ be a hash function. The signature scheme $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ obtained via the *Fiat-Shamir transformation with aborts* on the identification scheme IDS is defined as follows:

- $\text{KeyGen}(1^\lambda)$: Outputs $(pk, sk) \leftarrow \text{IDS.Gen}(1^\lambda)$.
- $\text{Sign}(sk, M)$:
 1. Generate an internal state r and compute commitment W .

2. Compute the challenge on W and the message M , that is, $c = \mathcal{H}(W \parallel M)$.
 3. Compute the response R .
 4. If $R = \perp$, return to step 1 (repeat at most κ_m times).
 5. Return the signature $\sigma = (W, R)$.
- **Verify**(pk, M, σ):
 1. Parse $\sigma = (W, R)$.
 2. Compute the challenge $c = \mathcal{H}(W \parallel M)$.
 3. Output 1 if $\text{IDS.V}(pk, \pi) = 1$, and 0 otherwise.

2 ATLAS DESIGN DESCRIPTIONS

We present the design rationale for ATLAS then describe the proposed ATLAS signature scheme. We first present an overview of the construction and the supporting algorithms. We then establish the correctness of the scheme. Next, we analyse the rejection sampling procedure and describe the parameter selection process. Finally, we evaluate the communication costs in terms of public-key and signature sizes.

2.1 Design Rationale

One of the primary design objectives of ATLAS is to achieve extreme parallelizability and a minimal memory footprint for resource-constrained IoT devices while maintaining high performance on larger platforms through module-space exploration. ATLAS bridges the gap between theoretical security and practical hardware constraints by selecting optimized design points in the lattice parameter space.

- **Module LWR:** The security of ATLAS is based on the hardness of the MLWR problem. By utilizing a deterministic rounding operation to implicitly generate errors, the scheme eliminates the need for explicit error sampling required in LWE-based designs. This significantly reduces the requirement for pseudo-random numbers, leading to better performance and smaller public-key sizes. Furthermore, the lack of explicit error sampling provides natural protection against certain fault-injection attacks that exploit seed manipulation during error vector generation [24].
- **Variable Polynomial Degree:** Unlike traditional designs that fix the polynomial degree at $n = 256$, ATLAS introduces a tiered approach where the degree scales with security ($n \in \{128, 256, 512\}$). This strategy provides a unique trade-off: lower security levels use $n = 128$ to save hardware resources (such as FPGA-based DSP units), while high-security instantiations can leverage $n = 512$ to achieve maximum post-quantum resistance. This flexibility allows for serial implementation to save memory or parallel instantiation for high-speed hardware security modules. A similar process has been explored to design efficient KEMs [19, 20].
- **Power-of-Two Moduli:** To ensure robust side-channel attack resilience, ATLAS utilizes power-of-two moduli (e.g., for ATLAS-128, $q = 2^{23}$ and $p = 2^{18}$). This eliminates the need for expensive modular reduction and rejection sampling in the primary arithmetic path. Specifically, this choice

facilitates efficient arbitrary-order masking by making arithmetically expensive operations, such as Arithmetic-to-Boolean conversions, significantly cheaper than in prime-modulus schemes [9, 18].

- **Small Polynomial Ring Size:** The use of varying polynomial sizes, starting at $n = 128$, offers two primary advantages. First, it ensures a *low memory footprint* on software platforms like the ARM Cortex-M4. Second, it enables *hardware parallelism*, as smaller multipliers require fewer DSP units, allowing hardware designers to instantiate multiple parallel units for matrix-vector multiplications without exhausting silicon area [20].
- **Polynomial Multiplication:** Since ATLAS uses power-of-two moduli and dimensions, it avoids the complexities of the Number Theoretic Transform (NTT). Instead, the scheme is optimized for Toom-Cook and Karatsuba-based multiplication. For the chosen degrees, these methods perform at par with NTT-based approaches while benefiting from free modular reduction, which is a significant advantage in hardware implementations.
- **SUF-CMA Security:** ATLAS is designed to ensure robust protection against existential forgeries. By utilizing the Fiat-Shamir with aborts paradigm and carefully selecting the bit-security bounds (ζ, ζ') , the scheme ensures that the signature remains secure against state-of-the-art lattice reduction attacks. The consistency of the rounding error ensures that even with truncated public keys ($pk_{drop} = 10$), the security levels of 128, 192, 256, and 512 bits are strictly maintained.
- **Optimisation of Storage and Transmission** The parameter sets for ATLAS are specifically selected to minimise the communication overhead. The rounding parameters p and pk_{drop} are chosen to compress the public key and signature data as much as possible without compromising the security margins (ζ, ζ') against state-of-the-art lattice reduction attacks.

2.2 ATLAS Specification

Our construction adopts the *Fiat-Shamir with Aborts* framework, employing a design methodology consistent with that of Dilithium [13]. In the following sections, we provide a concise overview of the signature’s core components, followed by an in-depth technical specification of each constituent algorithm.

Key Generation. $(pk, sk) \leftarrow \text{sig_keygen}(1^\lambda)$. The *key generation algorithm*, sig_keygen , starts by sampling a seed ρ and seed K , where ρ is used to deterministically generate the public matrix $\mathbf{A} \in \mathcal{R}_q^{k \times l}$. The secret component consists of a short vector $\mathbf{s}_1$ sampled uniformly from $\mathbf{S}_\eta^l$. The algorithm then computes the public vector $\mathbf{t} \leftarrow \lfloor \frac{p}{q} \mathbf{A} \mathbf{s}_1 \rfloor \in \mathcal{R}_p^k$. To achieve a compact public key, $\mathbf{t}$ is partitioned into $\mathbf{t}_1$ and $\mathbf{t}_0$ using a decomposition (decompose_keygen) at a power-of-two boundary 2^d , and publish $\mathbf{t}_1$ as a compressed version of $\mathbf{t}$ as public key. A collision-resistant hash function is used to compute $tr = \text{CRH}(\rho \parallel \mathbf{t}_1)$. The algorithm concludes by outputting the public key $pk = (\rho, \mathbf{t}_1)$ and the secret key $sk = (\rho, K, tr, \mathbf{s}_1, \mathbf{t}_0)$.

Signature Generation. $\sigma \leftarrow \text{sig_sign}(sk, M)$. The *signature algorithm* begins by reconstructing the public matrix $\mathbf{A}$ from the seed ρ and deriving the auxiliary secret component $\mathbf{s}_2 = \mathbf{t} - \frac{p}{q} \mathbf{A} \mathbf{s}_1$. A message-dependent digest μ is generated using the public key hash tr and the message M . The signer then enters an iterative loop where a masking vector $\mathbf{y}$ is sampled to produce a commitment $\mathbf{w} = \lfloor \frac{p}{q} \mathbf{A} \mathbf{y} \rfloor$, along with its associated rounding error ξ_1 . The challenge c is computed by hashing the high-order bits of the commitment, $\mathbf{w}_1$, together with μ . Following this, the potential response $\mathbf{z} = \mathbf{y} + c \mathbf{s}_1$ and a hint

Algorithm 1 `sig_keygen()`

-
- | | |
|--|--|
| 1: $\rho \xleftarrow{\$} \{0, 1\}^{256}, K \xleftarrow{\$} \{0, 1\}^{256}$ | ▷ Sampling seeds |
| 2: $\mathbf{A} \in \mathcal{R}_q^{k \times l} := \text{expand_mat}(\rho)$ and $\mathbf{s}_1 \xleftarrow{\$} \mathcal{S}_\eta^l$ | ▷ Generating $\mathbf{A}$ and $\mathbf{s}_1$ |
| 3: $\mathbf{t} := \left\lfloor \frac{p}{q} \mathbf{A} \mathbf{s}_1 \right\rfloor \in \mathcal{R}_p^k$ | ▷ Computing $\mathbf{t}$ |
| 4: $(\mathbf{t}_1, \mathbf{t}_0) := \text{decompose_keygen}(\mathbf{t}, 2^d)$ and $tr := \text{CRH}(\rho \parallel \mathbf{t}_1)$ | ▷ Compact public key |
| 5: $pk := (\rho, \mathbf{t}_1)$ and $sk = (\rho, K, tr, \mathbf{s}_1, \mathbf{t}_0)$ | |
| 6: return pk, sk | ▷ Output the key |
-

vector $\mathbf{h}$ are calculated. Here, $\mathbf{h}$ assists the verifier by compensating for rounding errors involving the public key component $\mathbf{t}_0$. To prevent information leakage and ensure verifiability, the algorithm performs two rejection sampling: at first, it verifies that the coefficients of $\mathbf{z}$ and $\mathbf{r}_0$ are within prescribed bounds, it means $\mathbf{z} \in \mathcal{S}_{\gamma_1 - \beta_1 - 1}^l$, and $\mathbf{r}_0 \in \mathcal{S}_{\bar{\gamma}_2 - \beta_2 - 1}^k$, and $\mathbf{r}_1 = \mathbf{w}_1$. It ensures that $\mathbf{z}$ does not leak any information about the secret vectors $\mathbf{s}_1, \mathbf{s}_2$. The second rejection sampling ensures that the Hamming weight (HW) of the hint vector $\mathbf{h}$ is less than ω and $\|\mathbf{ct}_0\|_\infty < \bar{\gamma}_2$. This ensures verifiability as mentioned. If these rejection sampling conditions are not met, the algorithm increments the counter and restarts. Upon success, the final signature $\sigma = (\mathbf{z}, \mathbf{h}, c)$ is returned.

Verification Algorithm. $0/1 \leftarrow \text{sig_verify}(pk, \sigma, M)$. The *verification algorithm*, `sig_verify`, determines the validity of a signature $\sigma = (\mathbf{z}, \mathbf{h}, c)$ for a message M . The verifier reconstructs the public matrix $\mathbf{A}$ and the message digest μ . Using the hint vector $\mathbf{h}$, the verifier reconstructs the high-order bits of the commitment, $\mathbf{w}'_1$, from the term $\left\lfloor \frac{p}{q} \mathbf{A} \mathbf{z} \right\rfloor - \mathbf{ct}_1 \cdot 2^{pk_{drop}}$. The signature is accepted if the challenge c matches the hash of the reconstructed bits and all norm constraints on $\mathbf{z}$ and $\mathbf{h}$ are satisfied.

2.3 Supporting Algorithms

The proposed signature scheme incorporates several auxiliary subroutines to manage the trade-off between precision and storage while ensuring correctness in the presence of rounding errors. The bit decomposition functions `decompose_keygen`, and `decompose_sign` partition a ring element $a \in \mathbb{Z}_q$ into a high-order component a_1 and a low-order remainder a_0 . This mechanism facilitates the generation of a compact public key $pk := (\rho, \mathbf{t}_1)$ during `KeyGen` and enables the signer to derive the challenge c from $\mathbf{w}_1$ during `Sign`. To compensate for the loss of precision caused by public key truncation, the hinting algorithms `make_hint` and `use_hint` identify and resolve potential carry-overs. Specifically, the signer employs `make_hint` to detect if the secret-key-dependent term $\mathbf{cs}_2$ alters the high-order bits of the commitment. This allows the verifier to use `use_hint` to reconstruct $\mathbf{w}'_1$ and successfully validate the signature. Note that the output of each `decompose` function is categorized as (high bits, low bits).

decompose_keygen. Algorithm 4 partitions a coefficient $a \in \mathbb{Z}_q$ into a high-order component a_1 and a low-order remainder a_0 such that $a \equiv a_1 \cdot 2^{pk_{drop}} + a_0 \pmod{q}$. The high part a_1 is computed via a biased rounding $a_1 = \left\lfloor \frac{a - 1 + 2^{pk_{drop} - 1}}{2^{pk_{drop}}} \right\rfloor$, which centres the decomposition to ensure a_0 remains small. The remainder is then derived as $a_0 = q + a - a_1 \cdot 2^{pk_{drop}}$.

Algorithm 2 $\text{sig_sign}(sk, M)$

```

1:  $\mathbf{A} \in \mathcal{R}_q^{k \times l} := \text{expand\_mat}(\rho)$  ▷ Reconstruct public matrix from seed
2:  $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$  ▷ Reassemble public vector from high/low bits
3:  $\mathbf{s}_2 := (\mathbf{t} - \frac{p}{q}\mathbf{A}\mathbf{s}_1)$  ▷ Reconstruct the full secret component
4:  $\mu := \text{CRH}(tr \parallel M)$  ▷ Hash public key and message
5:  $\text{count} := 0$  ▷ Initialize attempt counter
6: repeat ▷ Start of Rejection Sampling Loop
7:    $\text{count} \leftarrow \text{count} + 1$ 
8:    $\mathbf{y} \in \mathcal{S}_{\gamma_1-1}^l := \text{Sam}(K \parallel \mu \parallel \text{count})$  ▷ Sample masking vector
9:    $\mathbf{w} := \lfloor \frac{p}{q}\mathbf{A}\mathbf{y} \rfloor \in \mathcal{R}_p^k$  ▷ Compute commitment
10:   $\boldsymbol{\xi}_1 := \lfloor \frac{p}{q}\mathbf{A}\mathbf{y} \rfloor - \frac{p}{q}\mathbf{A}\mathbf{y}$  ▷ Capture rounding error
11:   $(\mathbf{w}_1, \_) := \text{decompose\_sign}(\mathbf{w}, 2\bar{\gamma}_2)$  ▷ Extract high-order bits
12:   $c := \text{H}(\mu \parallel \mathbf{w}_1) \in \mathcal{B}_\kappa$  ▷ Generate challenge
13:   $\mathbf{z} := \mathbf{y} + c\mathbf{s}_1$  ▷ Compute potential response
14:   $\boldsymbol{\xi}_2 := \lfloor c\mathbf{s}_2 \rfloor - c\mathbf{s}_2$  ▷ Calculate secondary error
15:   $\boldsymbol{\nu} := \lfloor \boldsymbol{\xi}_2 - \boldsymbol{\xi}_1 \rfloor$  ▷ Compute noise compensation
16:   $(\mathbf{r}_1, \mathbf{r}_0) := \text{decompose\_sign}(\mathbf{w} - \lfloor c\mathbf{s}_2 \rfloor + \boldsymbol{\nu}, 2\bar{\gamma}_2)$  ▷ Decompose adjusted commitment
17:   $\mathbf{h} := \text{makehint}(-c\mathbf{t}_0, \mathbf{w} - \lfloor c\mathbf{s}_2 \rfloor + \boldsymbol{\nu} + c\mathbf{t}_0, 2\bar{\gamma}_2)$  ▷ Generate hints
18:   $\text{accept} \leftarrow \text{True}$ 
19:  if  $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta_1$  or  $\|\mathbf{r}_0\|_\infty \geq \bar{\gamma}_2 - \beta_2$  or  $\mathbf{r}_1 \neq \mathbf{w}_1$  then
20:     $\text{accept} \leftarrow \text{False}$  ▷ Security and correctness check
21:  end if
22:  if  $\text{HW}(\mathbf{h}) > \omega$  or  $\|c\mathbf{t}_0\|_\infty \geq \bar{\gamma}_2$  then
23:     $\text{accept} \leftarrow \text{False}$  ▷ Hint and weight constraints
24:  end if
25: until  $\text{accept} = \text{True}$  ▷ Repeat if any condition fails
26:  $\sigma \leftarrow (\mathbf{z}, \mathbf{h}, c)$ 
27: return  $\sigma$ 

```

Algorithm 3 $\text{sig_verify}(pk, \sigma, M)$

```

1:  $\mathbf{A} \in \mathcal{R}_q^{k \times l} := \text{expand\_mat}(\rho)$  ▷ Reconstruct public matrix from seed
2:  $\mu \leftarrow \text{CRH}(\text{CRH}(pk) \parallel M)$  ▷ Compute message digest with public key hash
3:  $\mathbf{w}'_1 \leftarrow \text{use\_hint}(\mathbf{h}, \lfloor \frac{p}{q}\mathbf{A}\mathbf{z} \rfloor - c\mathbf{t}_1 \cdot 2^{p k_{drop}}, 2\bar{\gamma}_2)$  ▷ Reconstruct commitment high bits using hint
4:  $c' := \text{H}(\mu \parallel \mathbf{w}'_1)$  ▷ Generate candidate challenge
5: if  $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$  and  $c = c'$  and  $\text{HW}(\mathbf{h}) \leq \omega$  then ▷ Check norms, challenge, and hint weight
6:   return 1 ▷ Signature is valid
7: else
8:   return 0 ▷ Signature is invalid
9: end if

```

decompose_sign. Algorithm 5 decomposes a coefficient $a \in \mathbb{Z}_p$ into a high-order component a_1 and a low-order remainder a_0 such that $a \equiv a_1 \cdot 2^\delta + a_0 \pmod{p}$, where $\delta = \text{bits}_{\bar{\gamma}_1}$. The high part is computed using a biased rounding shift $a_1 = \lfloor \frac{a-1+\bar{\gamma}_2}{2^\delta} \rfloor$. This bias ensures that the resulting low part a_0 is centred, minimizing its magnitude. The remainder is defined as $a_0 = p + a - a_1 \cdot 2^\delta$ to ensure a positive representation in modulo p . Finally, the high part is constrained to the quotient space via

Algorithm 4 `decompose_keygen(a)`

```

1:  $a_1 := (a - 1 + 2^{pk_{drop}-1}) \ggg pk_{drop}$  ▷ Compute high bits with rounding offset
2:  $a_0 := q + a - (a_1 \lll pk_{drop})$  ▷ Compute low bits/remainder
3: return  $(a_1, a_0)$  ▷ Return decomposed components

```

Algorithm 5 `decompose_sign(a)`

```

1:  $a_1 := (a - 1 + \bar{\gamma}_2) \ggg \text{bits}_{\bar{\gamma}_1}$  ▷ Extract high bits with  $\bar{\gamma}_2$  offset
2:  $a_0 := p + a - (a_1 \lll \text{bits}_{\bar{\gamma}_1})$  ▷ Compute low bits with modulus adjustment
3:  $a_1 \leftarrow a_1 \& ((p \ggg \text{bits}_{\bar{\gamma}_1}) - 1)$  ▷ Apply mask for range reduction
4: return  $(a_1, a_0)$ 

```

$a_1 \leftarrow a_1 \bmod (\frac{p}{2^\delta})$, implemented efficiently as a bitwise AND: $a_1 \& ((p \ggg \delta) - 1)$. Bounding a_0 through this decomposition is necessary for both protocol correctness and the security of the abort condition.

make_hint. Algorithm 6 determines whether adding a value $b \in \mathbb{Z}_p$ to a coefficient $a \in \mathbb{Z}_p$ changes its high-bit decomposition. Let $\tilde{a} = (a + b) \bmod p$, which is implemented in the code as $(a + b) \& (p - 1)$. The function computes the high parts using the decomposition procedure, $(a_1, _) \leftarrow \text{decompose_sign}(a)$, and $(a_2, _) \leftarrow \text{decompose_sign}(\tilde{a})$. The output is defined as

$$\text{make_hint}(a, b) = \begin{cases} 1, & \text{if } a_1 \neq a_2, \\ 0, & \text{otherwise.} \end{cases}$$

Thus, the function outputs a single-bit hint indicating whether the addition of b causes a change in the high part of a . This hint is used in the signing procedure to assist in reconstructing the correct high bits after perturbation, while ensuring that no additional information about the secret is leaked.

use_hint. The Algorithm 7 reconstructs the correct high part of a coefficient using a one-bit hint. It takes $a \in \mathbb{Z}_p$ and $\text{hint} \in \{0, 1\}$ as input. First, the coefficient is reduced in modulo p and decomposed as $(a_1, a_0) \leftarrow \text{decompose_sign}(a \bmod p)$, such that $a \equiv a_1 \cdot 2^\delta + a_0 \pmod{p}$, where $\delta = \text{bits}_{\bar{\gamma}_1}$. The notation $\text{bits}_{\bar{\gamma}_1}$ denote the number of bits required to represent the value $\bar{\gamma}_1$, that is, $\lceil \log_2 \bar{\gamma}_1 \rceil$. If $\text{hint} = 0$, the function returns a_1 , indicating that no correction is needed. If $\text{hint} = 1$, the function adjusts the high part based on the sign of the low part a_0 . Since a_0 is represented in modulo p as $a_0 = p + (a - a_1 \cdot 2^\delta)$, values satisfying $a_0 < p$ correspond to negative representatives, while values $a_0 \geq p$ correspond to non-negative representatives. Therefore, the update rule is,

$$a_1 \leftarrow \begin{cases} (a_1 + 1) \bmod 2^{\delta_1}, & \text{if } a_0 > p, \\ (a_1 - 1) \bmod 2^{\delta_1}, & \text{otherwise,} \end{cases}$$

where $2^{\delta_1} = \frac{p}{2^\delta}$ and the modular reduction is implemented via bit-masking $\& ((p \ggg \delta) - 1)$. Thus, the function uses the hint to correct the high part by ± 1 , allowing recovery of the correct decomposition after perturbation during signing, while preserving security.

Algorithm 6 `make_hint(a, b)`

```

1:  $v := (a + b) \ \& \ (p - 1)$  ▷ Perform modular addition
2:  $a_1 \leftarrow \text{decompose\_sign}(a)$  ▷ Extract high bits of original value
3:  $a_2 \leftarrow \text{decompose\_sign}(v)$  ▷ Extract high bits of summed value
4: if  $a_1 \neq a_2$  then
5:   return 1 ▷ A hint is required to correct the high bits
6: else
7:   return 0
8: end if

```

Algorithm 7 `use_hint(a, hint)`

```

1:  $(a_1, a_0) \leftarrow \text{decompose\_sign}(a \ \& \ (p - 1))$  ▷ Decompose masked value into high/low bits
2: if  $hint = 0$  then
3:   return  $a_1$  ▷ No correction needed
4: end if
5:  $mask := (p \gg \text{bits}_{\tilde{\gamma}_1}) - 1$  ▷ Define modular range mask
6: if  $a_0 > p$  then
7:   return  $(a_1 + 1) \ \& \ mask$  ▷ Correct for underflow
8: else
9:   return  $(a_1 - 1) \ \& \ mask$  ▷ Correct for overflow
10: end if

```

2.4 Correctness of the Signature Scheme

A digital signature scheme is considered *correct* if an honestly generated signature is always accepted by the verification algorithm.

Definition 2.1 (Correctness). *A digital signature scheme $SIGN = (\text{KeyGen}, \text{Sign}, \text{Verify})$ is correct if for every security parameter λ , every message $\mu \in \{0, 1\}^*$, and every key pair $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, the following condition is satisfied:*

$$\Pr[\text{Verify}(pk, \mu, \sigma) = 1 \mid \sigma \leftarrow \text{Sign}(sk, \mu)] = 1$$

Based on the preceding definition, we now establish the correctness of the proposed signature scheme by demonstrating that an honestly generated signature satisfies the verification identity with probability 1.

Theorem 2.1 (Correctness). *The digital signature scheme $ATLAS = (\text{sig_keygen}, \text{sig_sign}, \text{sig_verify})$ is correct if, for any signature $\sigma = (\mathbf{z}, \mathbf{h}, c)$ generated by $\text{sig_sign}(sk, M)$, the reconstructed high-order bits $\mathbf{w}'_1$ satisfy $\mathbf{w}'_1 = \mathbf{w}_1$. This equality is guaranteed if the rejection sampling criteria in sig_sign ensure that the error terms associated with $\mathbf{s}_1, \mathbf{s}_2$, and $\mathbf{t}_0$ remain within the correction range of the `use_hint` function.*

Proof. To prove the correctness of the signature scheme, we evaluate the reconstruction of the high-order bits during verification. Given that $\mathbf{ct}$ contains only integer coefficients, we move the term inside the rounding operator to expand the relation:

$$\left\lceil \frac{p}{q} \mathbf{A} \mathbf{z} \right\rceil - \mathbf{ct} = \left\lceil \frac{p}{q} \mathbf{A} (\mathbf{y} + c \mathbf{s}_1) - \mathbf{ct} \right\rceil = \left\lceil \frac{p}{q} \mathbf{A} \mathbf{y} - c \left(\mathbf{t} - \frac{p}{q} \mathbf{A} \mathbf{s}_1 \right) \right\rceil. \quad (1)$$

By applying the definition of the secret key component $\mathbf{s}_2 = \mathbf{t} - \frac{p}{q}\mathbf{A}\mathbf{s}_1$ and incorporating the rounding errors $\boldsymbol{\xi}_1 = \lfloor \frac{p}{q}\mathbf{A}\mathbf{y} \rfloor - \frac{p}{q}\mathbf{A}\mathbf{y}$ and $\boldsymbol{\xi}_2 = \lfloor c\mathbf{s}_2 \rfloor - c\mathbf{s}_2$, the expression simplifies to:

$$\begin{aligned}
\left\lfloor \frac{p}{q}\mathbf{A}\mathbf{z} \right\rfloor - c\mathbf{t} &= \left\lfloor \frac{p}{q}\mathbf{A}\mathbf{y} - c\mathbf{s}_2 \right\rfloor \\
&= \left\lfloor \left\lfloor \frac{p}{q}\mathbf{A}\mathbf{y} \right\rfloor - \left\lfloor \frac{p}{q}\mathbf{A}\mathbf{y} \right\rfloor + \frac{p}{q}\mathbf{A}\mathbf{y} - \lfloor c\mathbf{s}_2 \rfloor + \lfloor c\mathbf{s}_2 \rfloor - c\mathbf{s}_2 \right\rfloor \\
&= \left\lfloor \left\lfloor \frac{p}{q}\mathbf{A}\mathbf{y} \right\rfloor - \boldsymbol{\xi}_1 - \lfloor c\mathbf{s}_2 \rfloor + \boldsymbol{\xi}_2 \right\rfloor \\
&= \left\lfloor \frac{p}{q}\mathbf{A}\mathbf{y} \right\rfloor - \lfloor c\mathbf{s}_2 \rfloor + \lfloor \boldsymbol{\xi}_2 - \boldsymbol{\xi}_1 \rfloor \\
\implies \left\lfloor \frac{p}{q}\mathbf{A}\mathbf{z} \right\rfloor - c\mathbf{t} &= \mathbf{w} - \lfloor c\mathbf{s}_2 \rfloor + \boldsymbol{\nu}, \tag{2}
\end{aligned}$$

where $\mathbf{w} = \lfloor \frac{p}{q}\mathbf{A}\mathbf{y} \rfloor$ and $\boldsymbol{\nu} = \lfloor \boldsymbol{\xi}_2 - \boldsymbol{\xi}_1 \rfloor$. Substituting the public key decomposition $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$, we derive the input to the hint function as $\lfloor \frac{p}{q}\mathbf{A}\mathbf{z} \rfloor - c\mathbf{t}_1 \cdot 2^d = \mathbf{w} - \lfloor c\mathbf{s}_2 \rfloor + \boldsymbol{\nu} + c\mathbf{t}_0$. Provided the noise remains within the supported boundaries of the hint mechanism, the verifier reconstructs $\mathbf{w}'_1 = \text{decompose_sign}(\mathbf{w} - \lfloor c\mathbf{s}_2 \rfloor + \boldsymbol{\nu}, \bar{\gamma}_2)$. Since the signing algorithm enforces the condition $\mathbf{r}_1 = \mathbf{w}_1$, the identity $\mathbf{w}'_1 = \mathbf{w}_1$ holds. This completes the proof. $\square$

2.5 Analysis of Failure Probability: Rejection Sampling

We analyse the rejection-sampling conditions in the signing algorithm to ensure the output signature $\mathbf{z}$ and the hint vector $\mathbf{h}$ do not leak information about the secret key. The rejection conditions, as defined in Algorithm 2, are summarised as follows:

- **Event₁ (Line 19):** $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$.
- **Event₂ (Line 19):** $\|\mathbf{r}_0\|_\infty < \bar{\gamma}_2 - \beta_2$.
- **Event₃ (Line 19):** $\|\lfloor c\mathbf{s}_2 \rfloor - \boldsymbol{\nu}\|_\infty \leq \beta_2$ (ensuring $\mathbf{r}_1 = \mathbf{w}_1$).
- **Event₄ (Line 22):** $\|c\mathbf{t}_0\|_\infty < \bar{\gamma}_2$.
- **Event₅ (Line 22):** $\text{HW}(\mathbf{h}) < \omega$.

Analysing Event₁. We begin by estimating the probability of **Event₁**, which is defined as.

$$\Pr[\text{Event}_1] = \Pr[\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1]. \tag{3}$$

To evaluate $\Pr[\text{Event}_1]$, we analyze the distribution of the coefficients of $\mathbf{z}$. Recall from Line 19 of Algorithm 2 that $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, where $\mathbf{y} \in \mathbf{S}_{\gamma_1-1}^l$. Thus, each coefficient of the element $(\mathbf{y})_i$ is uniformly distributed over an interval of size $2\gamma_1 - 1$, where $(\mathbf{y})_i$ denotes the i -th element of $\mathbf{y}$. Let $\bar{k}$ denote a coefficient of an element of $c\mathbf{s}_1$. The condition $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$ implies that every coefficient of $\mathbf{z}$ lies

in the interval $(-\gamma_1 + \beta_1 + 1, \gamma_1 - \beta_1 - 1]$. Using $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, this is equivalent to requiring that the corresponding coefficient of $(\mathbf{y})_i$ belongs to $(-\gamma_1 + \beta_1 + 1 - \bar{k}, \gamma_1 - \beta_1 - 1 - \bar{k}]$, which contains exactly $2(\gamma_1 - \beta_1) - 1$ admissible values. Therefore, the probability that a coefficient satisfies the condition is $\frac{2(\gamma_1 - \beta_1) - 1}{2\gamma_1 - 1}$. Assuming independence across the nl coefficients, we obtain the following expression. For sufficiently large γ_1 , the value can be approximated as $\Pr[\text{Event}_1] = \exp(-nl\beta_1/\gamma_1)$.

$$\Pr[\text{Event}_1] = \Pr[\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1] = \left(\frac{2(\gamma_1 - \beta_1) - 1}{2\gamma_1 - 1} \right)^{nl} = \left(1 - \frac{\beta_1}{\gamma_1 - 1/2} \right)^{nl} \approx \exp(-nl\beta_1/\gamma_1). \quad (4)$$

Analysing Event₂. We next estimate the probability of Event_2 , which is defined as

$$\Pr[\text{Event}_2] = \Pr[\|\mathbf{r}_0\|_\infty < \bar{\gamma}_2 - \beta_2]. \quad (5)$$

Following the same analysis as for Event_1 , we obtain the following approximation for sufficiently large $\bar{\gamma}_2$ and $\beta_2 \gg 1/2$.

$$\Pr[\text{Event}_2] \approx \exp(-nk\beta_2/\bar{\gamma}_2)$$

Analysing Event₃. We analyze the condition $\mathbf{r}_1 = \mathbf{w}_1$. Recall that Line 11 computes $(\mathbf{w}_1, _) := \text{decompose_sign}(\mathbf{w}, 2\bar{\gamma}_2)$, and Line 16 computes $(\mathbf{r}_1, _) := \text{decompose_sign}(\mathbf{w} - \lfloor c\mathbf{s}_2 \rfloor + \boldsymbol{\nu}, 2\bar{\gamma}_2)$. The equality $\mathbf{r}_1 = \mathbf{w}_1$ holds whenever the term $\lfloor c\mathbf{s}_2 \rfloor - \boldsymbol{\nu}$ does not alter the high-order component. By the correctness of decompose_sign , this is guaranteed if $\|\lfloor c\mathbf{s}_2 \rfloor - \boldsymbol{\nu}\|_\infty \leq \beta_2$ and $\|\mathbf{r}_0\|_\infty < \bar{\gamma}_2 - \beta_2$. The latter condition has already been analyzed through Event_2 . Therefore, it remains to estimate the probability of $\text{Event}_3 := \{\|\lfloor c\mathbf{s}_2 \rfloor - \boldsymbol{\nu}\|_\infty \leq \beta_2\}$, given by

$$\Pr[\text{Event}_3] = \Pr[\|\lfloor c\mathbf{s}_2 \rfloor - \boldsymbol{\nu}\|_\infty \leq \beta_2]. \quad (6)$$

By definition, $\|\boldsymbol{\nu}\|_\infty \leq 1$, we can simplify as $\|\lfloor c\mathbf{s}_2 \rfloor\|_\infty \leq \beta_2 - 1$. Defining $\bar{\beta}_2 := \beta_2 - 1$, we rewrite the event as $\text{Event}_3 = \{\|\lfloor c\mathbf{s}_2 \rfloor\|_\infty \leq \bar{\beta}_2\}$. To estimate $\Pr[\text{Event}_3]$, we analyze the distribution of the coefficients of $c\mathbf{s}_2$. Let V denote a coefficient of $c\mathbf{s}_2$, and let U_i denote a coefficient of the i -th component of $\mathbf{s}_2$, where $\mathbf{s}_2 \in \mathbf{S}_{1/2}^k$. We model the random variables $U_1, U_2, \dots, U_k$ as independent and identically distributed (*i.i.d.*), with $U_i \sim \mathcal{U}(-\frac{1}{2}, \frac{1}{2})$. Consequently, $\text{Var}(U_i) = \frac{(\frac{1}{2} - (-\frac{1}{2}))^2}{12} = \frac{1}{12}$. Since each coefficient of c contains exactly κ non-zero entries, a coefficient of $c\mathbf{s}_2$ can be viewed as the sum of κ independent random variables distributed as U_i . Therefore, by the Central Limit Theorem, for sufficiently large κ , the random variable V can be approximated by $V \sim \mathcal{N}(0, \kappa')$, where $\kappa' = \kappa \cdot \text{Var}(U_i) = \frac{\kappa}{12}$. Using this approximation, the probability of Event_3 can be estimated as

$$\Pr[\text{Event}_3] = \Pr[\|\lfloor c\mathbf{s}_2 \rfloor\|_\infty < \bar{\beta}_2] \approx \left(1 - 2F_{\mathcal{N}(0, \kappa')}(-\bar{\beta}_2) \right)^{nk}. \quad (7)$$

Here $F_{\mathcal{N}(0, \kappa')}$ denotes the cumulative distribution function (CDF) of the Gaussian distribution $\mathcal{N}(0, \kappa')$.

For ATLAS, the parameter β_2 is selected so that the event $\|\lfloor c\mathbf{s}_2 \rfloor\|_\infty < \bar{\beta}_2$ occurs with overwhelming probability. In particular, the target acceptance probability is chosen to be at least $(1 - 2^{-10})$.

Analysing Event₄. We calculate the probability of the event Event_4 . The public-key component $\mathbf{t}$ is decomposed as $\mathbf{t} = \mathbf{t}_1 \cdot 2^{pk_{drop}} + \mathbf{t}_0$, and $\|\mathbf{t}_0\|_\infty \leq 2^{pk_{drop}} - 1$. Here each coefficient U_i of $\mathbf{t}_0$ is modeled as i.i.d. $\mathcal{U}(-2^{pk_{drop}-1}, 2^{pk_{drop}-1})$ with $\text{Var}(U_i) = \frac{2^{2pk_{drop}}}{12}$.

Each coefficient V of $c\mathbf{t}_0$ is the sum of κ independent variables distributed as U_i . By the Central Limit Theorem, for sufficiently large κ , V is approximated by a Gaussian distribution $V \sim \mathcal{N}(0, \text{Var}(V))$,

where $\text{Var}(V) = \kappa \cdot \text{Var}(U_i) = \frac{\kappa \cdot 2^{pk_{drop}}}{12}$. The probability that a single coefficient satisfies $|V| < \bar{\gamma}_2$ is $1 - 2F_{\mathcal{N}(0, \text{Var}(V))}(-\bar{\gamma}_2)$, where $F_{\mathcal{N}(0, \text{Var}(V))}$ denotes the CDF of $\mathcal{N}(0, \text{Var}(V))$. Assuming independence across all nk coefficients, we obtain:

$$\Pr[\text{Event}_4] \approx \left(1 - 2F_{\mathcal{N}(0, \text{Var}(V))}(-\bar{\gamma}_2)\right)^{nk}. \quad (8)$$

For **ATLAS**, parameters are selected such that $\kappa \cdot 2^{pk_{drop}-1} < 2\bar{\gamma}_2$. Given the bound $\|\mathbf{ct}_0\|_\infty \leq \|c\|_1 \cdot \|\mathbf{t}_0\|_\infty$, this selection ensures that the tail probability $F_{\mathcal{N}(0, \text{Var}(V))}(-\bar{\gamma}_2)$ remains negligible. Consequently, $\Pr[\text{Event}_4]$ is negligibly close to one.

Analysing Event₅. To estimate this probability $\Pr[\text{Event}_5]$, we first analyze the distribution of a single coefficient of $\mathbf{h}$. Let U_1 and U_2 denote coefficients of $\mathbf{r}_0$ and $\mathbf{ct}_0$, respectively, and define $V := U_1 + U_2$. By construction, the coefficient h is determined by the output of the decomposition procedure. More precisely, a hint bit is generated whenever the perturbation caused by V changes the high-order component produced by `decompose_sign`. Equivalently,

$$h = \begin{cases} 1, & \text{if } |V| > \bar{\gamma}_2, \\ 0, & \text{otherwise.} \end{cases}$$

Therefore, the first step is to estimate the probability $\Pr[h = 1] = \Pr[|V| > \bar{\gamma}_2]$. From the analysis of **Event₂** and **Event₄**, we have already established that $\|\mathbf{r}_0\|_\infty < \bar{\gamma}_2 - \beta_2$, and $\|\mathbf{ct}_0\|_\infty < \bar{\gamma}_2$. Consequently, we assume $U_1 \sim \mathcal{U}(-(\bar{\gamma}_2 - \beta_2), \bar{\gamma}_2 - \beta_2)$. Let f_{U_1} and f_{U_2} denote the probability density functions of U_1 and U_2 , respectively. Since $V = U_1 + U_2$, the density of V is as follows:

$$\begin{aligned} f_V(v) &= \int_{v-(\bar{\gamma}_2-\beta_2)}^{v+(\bar{\gamma}_2-\beta_2)} f_{U_1}(v-u_2) f_{U_2}(u_2) du_2 \\ &= \frac{1}{2(\bar{\gamma}_2 - \beta_2)} \int_{v-(\bar{\gamma}_2-\beta_2)}^{v+(\bar{\gamma}_2-\beta_2)} f_{U_2}(u_2) du_2 \\ &= \frac{1}{2(\bar{\gamma}_2 - \beta_2)} \left(F_{U_2}(v + (\bar{\gamma}_2 - \beta_2)) - F_{U_2}(v - (\bar{\gamma}_2 - \beta_2)) \right), \end{aligned}$$

where F_{U_2} denotes the cumulative distribution function of U_2 .

Using the above density, the probability $\Pr[h = 1]$ can be estimated by evaluating the tail probability $\Pr[|V| > \bar{\gamma}_2]$. Subsequently, assuming independence across coefficients, the distribution of $\text{HW}(\mathbf{h})$ can be approximated by a binomial distribution, which yields an estimate for $\Pr[\text{Event}_5]$.

We next derive the CDF of the random variable V . Integrating the density function obtained above yields

$$F_V(v) = \int_{-\infty}^v f_V(u) du = \frac{1}{2(\bar{\gamma}_2 - \beta_2)} \int_{v-(\bar{\gamma}_2-\beta_2)}^{v+(\bar{\gamma}_2-\beta_2)} F_{U_2}(u) du. \quad (9)$$

Here, f_{U_1} , f_{U_2} , and f_V denote the probability density functions of U_1 , U_2 , and V , respectively, while F_{U_1} , F_{U_2} , and F_V denote their corresponding cumulative distribution functions. Using the symmetry of V , the probability that a coefficient of the hint vector is non-zero can be expressed as

$$p_h = \Pr[|V| > \bar{\gamma}_2] = 2F_V(-\bar{\gamma}_2) = \frac{1}{\bar{\gamma}_2 - \beta_2} \int_{-2(\bar{\gamma}_2-\beta_2)}^0 F_{U_2}(u) du. \quad (10)$$

From the analysis of **Event**₄, the random variable U_2 is approximated by a Gaussian distribution with variance $\text{Var}(U_2) = 2^{2p_{k_{drop}}} \cdot \kappa'$, where $\kappa' = \kappa/12$. Consequently,

$$p_h = \frac{1}{\bar{\gamma}_2 - \beta_2} \int_{-2(\bar{\gamma}_2 - \beta_2)}^0 F_{\mathcal{N}(0, 2^{2p_{k_{drop}}} \kappa')} (u) du. \quad (11)$$

Having estimated p_h , we now analyze the distribution of the Hamming weight of the hint vector $\mathbf{h}$. Since each coefficient contributes independently to the hint generation process, the random variable $\text{HW}(\mathbf{h})$ can be approximated by a binomial distribution $\text{HW}(\mathbf{h}) \sim \mathcal{B}(nk, p_h)$. Let $F_{\mathcal{B}(nk, p_h)}$ denote the corresponding cumulative distribution function. Then the probability of **Event**₅ can be approximated by

$$\Pr[\text{Event}_5] = \Pr[\text{HW}(\mathbf{h}) < \omega] \approx F_{\mathcal{B}(nk, p_h)}(\omega). \quad (12)$$

For **ATLAS**, the parameter ω is selected so that the event $\text{HW}(\mathbf{h}) < \omega$ holds with overwhelming probability. In particular, the target acceptance probability is chosen to be at least $1 - 2^{-10}$.

Analysing the whole event **Event_{RS}.** Combining the above analysis, we saw that $\Pr[\text{Event}_3]$, $\Pr[\text{Event}_4]$, and $\Pr[\text{Event}_5]$ are all negligibly close to one. Consequently, the overall acceptance probability is dominated by **Event**₁ and **Event**₂. Therefore,

$$\Pr[\text{Event}_{\text{RS}}] \approx \Pr[\text{Event}_1] \cdot \Pr[\text{Event}_2] \approx \exp\left(-n \left(\frac{\beta_1 l}{\gamma_1} + \frac{\beta_2 k}{\bar{\gamma}_2} \right)\right). \quad (13)$$

Hence, the expected number of iterations of the rejection-sampling loop is approximately

$$\frac{1}{\Pr[\text{Event}_{\text{RS}}]} = \exp\left(n \left(\frac{\beta_1 l}{\gamma_1} + \frac{\beta_2 k}{\bar{\gamma}_2} \right)\right). \quad (14)$$

2.6 Criteria for Parameter Selection

This section presents the parameter selection process for **ATLAS**. The specification defines four parameter sets, namely **ATLAS**-128, **ATLAS**-192, **ATLAS**-256, and **ATLAS**-512, targeting classical security levels of 128, 192, 256, and 512 bits, respectively. The parameter sets **ATLAS**-128, **ATLAS**-256, and **ATLAS**-512 follow the NGCC security recommendations, whereas **ATLAS**-192 is included as an *optional set* of intermediate-security parameters. We present these parameters in Table 2.

Choice of moduli: q and p . For **ATLAS**, we select $q = 2^{23}$ for **ATLAS**-128, **ATLAS**-192, and **ATLAS**-256, and $q = 2^{25}$ for **ATLAS**-512. The corresponding values of the rounding modulus p are 2^{18} , 2^{19} , 2^{20} , and 2^{22} , respectively. The use of power-of-two moduli allows the rounding procedure to be realised through bit-shift operations, resulting in a simple and efficient implementation. This design philosophy is also employed by the Module-LWR-based KEM Saber [12].

Choice of the ring degree n . The ring degree n is chosen to achieve the targeted security level while minimizing implementation costs. Since a smaller ring degree generally results in lower computational complexity, reduced memory requirements, and more hardware-friendly implementations, we select the smallest feasible value of n for each parameter set. Based on our security analysis, we choose $n = 128$ for **ATLAS**-128 and **ATLAS**-192, $n = 256$ for **ATLAS**-256, and $n = 512$ for **ATLAS**-512.

Matrix dimensions (k, l) . The dimensions of the matrix (k, l) are set to $(9, 6)$ for ATLAS-128, $(13, 10)$ for ATLAS-192, $(8, 7)$ for ATLAS-256 and $(8, 7)$ for ATLAS-512. These choices are based on the security and efficiency trade-offs identified during the parameter exploration process.

Noise parameter η . The noise parameter η is set as $\eta := \left\lceil \frac{q}{2p} \right\rceil$. For each parameter set, the value of η is derived from the corresponding choices of q and p . For example, ATLAS-128 yields $\eta = 2^4$. The resulting noise distribution is incorporated into the security analysis (see Section 3.3), where bit security is estimated using the corresponding noise variance together with the selected values of (k, l) and n using the LWE estimator.

Challenge size κ and challenge set ChSet. Let κ denote the challenge size. The hash function H maps its input to a challenge polynomial in the set $\text{ChSet} \subseteq \mathcal{R}$. Each challenge polynomial contains exactly κ non-zero coefficients, where every non-zero coefficient is either -1 or 1 , and all remaining coefficients are 0 . Consequently, the size of the challenge set is $|\text{ChSet}| = 2^\kappa \binom{n}{\kappa}$.

The value of κ is selected according to the ring degree n and the targeted security level. Specifically, $\kappa = 31$ for ATLAS-128 and $\kappa = 60$ for ATLAS-256. More generally, for each parameter set, κ is chosen such that the size of the challenge set exceeds the targeted security level. For example, for ATLAS-256, where $n = 256$ and $\kappa = 60$, the challenge set satisfies $|\text{ChSet}| > 2^{256}$.

Parameters β_1 and β_2 . The values of β_1 and β_2 are chosen to satisfy the bounds $\|\mathbf{cs}_1\|_\infty < \beta_1$, and $\|\mathbf{cs}_2\|_\infty < \beta_2 - 1 (= \beta'_2)$ (The detailed derivation of these bounds is provided in the previous subsection). Using $\|\mathbf{cs}_1\|_\infty \leq \|c\|_1 \cdot \|\mathbf{s}_1\|_\infty \approx \kappa\eta$, i.e., the value of β_1 is chosen to be approximately the multiplication of the selected values of κ and η , however due to requirement on β_2 the calculation of β_1 becomes complicated. The parameter β_2 is chosen such that the value of $n \left(\frac{\beta_1 l}{\gamma_1} + \frac{\beta_2 k}{\bar{\gamma}_2} \right)$ is approximately 1, this requirement forces the condition $\beta_1 \approx \kappa\eta$ to no longer be true because β_2 is required to be a positive integer.

Parameters $\gamma_1, \bar{\gamma}_1, \gamma_2$, and $\bar{\gamma}_2$. Set $\gamma_1 = \frac{q}{16}$, $\bar{\gamma}_1 = \frac{p}{q}\gamma_1$, $\gamma_2 = \frac{\gamma_1}{2}$, and $\bar{\gamma}_2 = \frac{\bar{\gamma}_1}{2}$. The value of γ_1 is fixed as $\frac{q}{16}$, and the value of $\bar{\gamma}_1$ is defined such that calling of `decompose_sign`, after the rounding operation, uses proper adjusted value of γ_1 , i.e., $\bar{\gamma}_1 = \frac{p}{q}\gamma_1$, since rounding operation involves multiplying by $\frac{p}{q}$. These values determine the acceptance bounds used in the rejection sampling procedure.

Setting of pk_{drop} . The parameter pk_{drop} specifies the number of least significant bits discarded from the public-key component $\mathbf{t}$. Its value is chosen to reduce the public-key size while ensuring that the resulting truncation error remains within the verification bounds. Let $\mathbf{t}_0$ denote the dropped part of $\mathbf{t}$. Since $\|\mathbf{ct}_0\|_\infty \leq \kappa \cdot 2^{pk_{drop}-1}$, the parameter pk_{drop} is selected such that $\kappa (2^{pk_{drop}-1} + 1) < 2\bar{\gamma}_2 - 1$. This condition guarantees that the contribution of the discarded bits is absorbed by the verification margin and does not affect the correctness of the scheme.

Choice of ω . We mentioned in Section 2.6 that ATLAS, the parameter ω is selected so that event $\text{HW}(\mathbf{h}) < \omega$ occurs with overwhelming probability. We set ω to be 128 for all ATLAS parameter sets.

Table 2: ATLAS Parameter Sets for Different Security Levels

Parameter Set	ATLAS-128	ATLAS-192	ATLAS-256	ATLAS-512
Classical Security (λ)	128 bits	192 bits	256 bits	512 bits
Quantum Security (Λ)	80 bits	96 bits	128 bits	256 bits
n (Ring Degree)	128	128	256	512
(q, p) (Modulus)	$(2^{23}, 2^{18})$	$(2^{23}, 2^{19})$	$(2^{23}, 2^{20})$	$(2^{25}, 2^{22})$
k, l (Dimensions)	$(9, 6)$	$(13, 10)$	$(8, 7)$	$(8, 7)$
η (noise parameter)	16	8	4	4
κ (challenge size)	31	69	60	60
(β_1, β_2) (Bounds)	$(51, 8)$	$(33, 12)$	$(73, 12)$	$(73, 12)$
$(\gamma_1, \gamma_2, \bar{\gamma}_1, \bar{\gamma}_2)$	$(2^{19}, 2^{18}, 2^{14}, 2^{13})$	$(2^{19}, 2^{18}, 2^{15}, 2^{14})$	$(2^{19}, 2^{18}, 2^{16}, 2^{15})$	$(2^{21}, 2^{20}, 2^{18}, 2^{17})$
pk_{drop}	10	10	10	10
ω (Weight)	128	128	128	128

2.7 Communication Cost: Key and Signature Size

2.7.1 Key Sizes

The storage requirements for public and secret keys are optimized for **ATLAS**. Due to the deterministic nature of the LWR problem, the secret key size is significantly reduced.

Public Key. The public key $pk = (\rho, \mathbf{t}_1)$ has a total size of $\left(\frac{n}{8} \cdot k \cdot (\lceil \log p \rceil - pk_{drop} + 1) + 32\right)$ bytes. In our construction, we select the truncation parameter d such that $\lceil \log(\kappa \cdot 2^{pk_{drop}-1}) \rceil = \log(2\gamma_2)$, which implies $pk_{drop} := \log(2\gamma_2) - \log(\kappa) + 1$. Consequently, the bit-length of $\mathbf{t}_1$ is $(\log p - \log(2\gamma_2) + \log(\kappa) - 1)$.

Secret Key. The secret key $sk = (\rho, K, tr, \mathbf{s}_1, \mathbf{t}_0)$ comprises $\left(112 + \frac{n}{8} \cdot (l \cdot \lceil \log(2\eta + 1) \rceil + pk_{drop} \cdot k)\right)$ bytes. The value of $\lceil \log(2\eta + 1) \rceil$ is fixed as 6 for all parameters in the implementation, since 6 is the value required to store the largest value of η (which is 16). **ATLAS** achieves a significant reduction in secret key size compared to **Dilithium** for two primary reasons. First, the noise vector $\mathbf{s}_2$ does not require explicit storage; because our construction is based on LWR, $\mathbf{s}_2$ can be deterministically reconstructed during the **sig_sign** procedure. Second, the modulus is reduced from q to p , hence the bit-width pk_{drop} is smaller than the fixed value of 14 used in **Dilithium**. Compared to the secret key sizes in **Dilithium**, our approach reduces the storage overhead by approximately 26% to 34%.

2.7.2 Signature Size

The signature $\sigma = (\mathbf{z}, \mathbf{h}, c)$ is calculated as $\left(\frac{n}{8} \cdot l \cdot \log(2\gamma_1) + \omega + k + \frac{n}{8} + 8\right)$ bytes for 128-bit, 192-bit and 256-bit security levels and as $\left(\frac{n}{8} \cdot l \cdot \log(2\gamma_1) + \omega \cdot \frac{\log n}{8} + k + \frac{n}{8} + 8\right)$ for 512-bit security level. This size is consistent with Dilithium benchmarks, as the parameters for the masking vector bound γ_1 , the challenge weight β_1 , and the number of non-zero hint bits ω are maintained at similar scales to ensure competitive performance and security.

Table 3: Communication Cost: Mathematical Expressions for Key and Signature Sizes. The formula given for the signature sizes is the 512-bit security level and not for the other security levels (see 2.7.2)

Component	Mathematical Expression (Bytes)
Public Key (pk)	$\frac{n}{8} \cdot k \cdot (\lceil \log p \rceil - pk_{drop} + 1) + 32$
Secret Key (sk)	$112 + \frac{n}{8} \cdot (l \cdot 6 + pk_{drop} \cdot k)$
Signature (σ)	$\frac{n}{8} \cdot l \cdot \log(2\gamma_1) + \omega \cdot \frac{\log(n)}{8} + k + \frac{n}{8} + 8$

For the ATLAS-128, substituting $n = 128$, $k = 9$, $l = 6$, $p = 2^{18}$, $\eta = 16$, $\gamma_1 = 2^{19}$, $pk_{drop} = 10$, and $\omega = 128$ into the expressions above yields a public key size of 1328 bytes, a secret key size of 2128 bytes, and a signature size of 2081 bytes. The corresponding sizes for all security levels are listed in Table 1.

3 SECURITY ARGUMENTS

This section establishes the formal security of the ATLAS signature scheme. The construction follows the *Fiat-Shamir with Abort* paradigm. In Section 3.2, we demonstrate that the scheme is existentially unforgeable against chosen-message attacks (EUF-CMA) based on the hardness of two underlying lattice-based problems. Finally, we analyse the best-known computational complexity for solving these problems using both classical and quantum algorithms.

3.1 Hard Problems

Definition 3.1 (MLWR PROBLEM). Let $\mathbf{pp} = (k, l, S)$ be the problem parameters. Let $\mathcal{U}$ denote the uniform distribution over $\mathcal{R}_q^{k \times l} \times \mathcal{R}_p^k$. The $\text{MLWR}_{\mathbf{pp}}$ distribution samples $(\mathbf{A}, \mathbf{t} = \lfloor \frac{p}{q} \mathbf{A} \mathbf{s} \rfloor)$, where $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}$ and $\mathbf{s} \xleftarrow{\$} S^l \subset \mathcal{R}_q^l$. The advantage of a distinguisher $\mathcal{D}$ for the decisional MLWR problem is:

$$\text{Adv}_{\text{MLWR}}^{\mathbf{pp}}(\mathcal{D}) := \left| \Pr \left[\mathcal{D}(\mathbf{A}, \mathbf{t}) = 1 \mid (\mathbf{A}, \mathbf{t}) \xleftarrow{\$} \mathcal{U} \right] - \Pr \left[\mathcal{D}(\mathbf{A}, \mathbf{t}) = 1 \mid (\mathbf{A}, \lfloor \frac{p}{q} \mathbf{A} \mathbf{s} \rfloor) \in \text{MLWR}_{\mathbf{pp}} \right] \right|$$

Definition 3.2 (MSIS PROBLEM). Given a problem parameters $\mathbf{pp} = \{k, l, \gamma\}$, and a matrix $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}$, the $\text{MSIS}_{\mathbf{pp}}$ problem requires finding a vector $\mathbf{v} \in \mathcal{R}^{k+l}$ such that $0 < \|\mathbf{v}\|_{\infty} \leq \gamma$ and $[\mathbf{I}_k \mid \mathbf{A}] \cdot \mathbf{v} = \mathbf{0}_k \in \mathcal{R}_q^k$. The advantage of an adversary $\mathcal{A}$ is:

$$\text{Adv}_{\text{MSIS}}^{\mathbf{pp}}(\mathcal{A}) := \Pr \left[0 < \|\mathbf{v}\|_{\infty} \leq \gamma \wedge [\mathbf{I}_k \mid \mathbf{A}] \cdot \mathbf{v} = \mathbf{0}_k \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}, \mathbf{v} \leftarrow \mathcal{A}(\mathbf{A}) \right]$$

Definition 3.3 (SelfTargetMSIS PROBLEM). Let $H : \{0, 1\}^* \rightarrow B_\kappa$ be a cryptographic hash function, $\text{pp} = (H, k, l, \zeta)$. Given a matrix $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}$, the $\text{SelfTargetMSIS}_{\text{pp}}$ problem requires finding a message μ and a vector $\mathbf{v}$ such that $0 < \|\mathbf{v}\|_\infty \leq \gamma$ and $H(\mu \parallel [\mathbf{I}_k \mid \mathbf{A}] \cdot \mathbf{v}) = c$. The advantage of an adversary $\mathcal{A}$ is:

$$\text{Adv}_{\text{SelfTargetMSIS}}^{\text{pp}}(\mathcal{A}) := \Pr \left[0 < \|\mathbf{v}\|_\infty \leq \gamma \wedge H(\mu \parallel [\mathbf{I}_k \mid \mathbf{A}] \cdot \mathbf{v}) = c \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}; (\mathbf{v}, \mu) \leftarrow \mathcal{A}^{H(\cdot)}(\mathbf{A}) \right]$$

Classical hardness of SelfTargetMSIS problem. The classical reduction from the MSIS problem to the SelfTargetMSIS problem is well-established in the literature [13]. The reduction strategy employs the forking lemma through a rewinding technique. Following the approach in [13], suppose an adversary $\mathcal{A}$ accesses H as a random oracle. The adversary makes queries $H(\mu_1 \parallel \mathbf{w}_1), \dots, H(\mu_k \parallel \mathbf{w}_k)$ and receives uniformly random responses $c_1, \dots, c_k$. Eventually, $\mathcal{A}$ outputs a message μ_i and a vector $\mathbf{v}$ such that $H(\mu_i \parallel [\mathbf{I} \mid \mathbf{A}] \cdot \mathbf{v}) = c_i$ for some index $1 \leq i \leq k$. The reduction proceeds by rewinding $\mathcal{A}$ to the oracle query $H(\mu_i \parallel \mathbf{w}_i)$ and injecting a fresh, independent challenge $c'_i \neq c_i$. According to the forking lemma, if $\mathcal{A}$ is a successful forger, there is a non-negligible advantage (approximately $1/k$) that it will generate a second valid signature $\mathbf{v}'$ for the same message and commitment. Since both signatures satisfy the verification equation $[\mathbf{I} \mid \mathbf{A}] \cdot \mathbf{v} = [\mathbf{I} \mid \mathbf{A}] \cdot \mathbf{v}'$, we can exploit the linear relationship to find that $[\mathbf{I} \mid \mathbf{A}] \cdot (\mathbf{v} - \mathbf{v}') = \mathbf{0}$. The distinctness of the challenges ($c_i \neq c'_i$) guarantees that the difference $\mathbf{v} - \mathbf{v}'$ is a non-zero vector. Because both $\mathbf{v}$ and $\mathbf{v}'$ have small coefficients, their difference remains sufficiently short, effectively yielding a solution to the MSIS problem.

3.2 Formal Security

The standard security objective for digital signature schemes is *Existential Unforgeability under Chosen Message Attack* (EUF-CMA). In this model, an adversary is provided with the public key and granted adaptive access to a signing oracle. The adversary's goal is to produce a valid signature for a message not previously queried. A more stringent requirement is *Strong Unforgeability* (SUF-CMA), which considers the adversary successful if they produce a different signature for a message that has already been signed by the oracle.

Evaluating the ATLAS scheme's security in a post-quantum context requires moving beyond the classical Random Oracle Model (ROM) into the Quantum Random Oracle Model (QROM). While classical reductions to MLWR and MSIS rely on the Forking Lemma, this technique is fundamentally incompatible with quantum adversaries. Because a quantum forger may query the hash function in a superposition of states, the act of *rewinding* to extract a second signature is hindered by the no-cloning theorem and the collapse of the quantum state upon measurement. Consequently, proving SUF-CMA security for ATLAS in the QROM requires alternative reduction strategies that do not depend on simple rewinding.

Definition 3.4 (SUF-CMA SECURITY). A digital signature scheme $\text{DS} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ is *Strongly Unforgeable under Chosen Message Attack* (SUF-CMA) if for any polynomial-time adversary $\mathcal{A}$, the advantage $\text{Adv}_{\text{DS}}^{\text{SUF-CMA}}(\mathcal{A})$ is negligible. The security property is defined via the following experiment:

Setup. The challenger runs $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ and gives the public key pk to the adversary $\mathcal{A}$.

Query Phase. The adversary $\mathcal{A}$ is given access to a signing oracle $\text{OSign}(sk, \cdot)$. $\mathcal{A}$ may adaptively submit messages μ_i and receive valid signatures σ_i . Let $\mathcal{Q} = \{(\mu_1, \sigma_1), (\mu_2, \sigma_2), \dots, (\mu_\ell, \sigma_\ell)\}$ be the set of message-signature pairs obtained.

Forgery. $\mathcal{A}$ outputs a forgery (μ^*, σ^*) . The adversary wins the experiment if: $\text{Verify}(pk, \mu^*, \sigma^*) = 1$ and $(\mu^*, \sigma^*) \notin \mathcal{Q}$

The advantage of the adversary is defined as:

$$\text{Adv}_{\text{DS}}^{\text{SUF-CMA}}(\mathcal{A}) := \Pr[\text{Verify}(pk, \mu^*, \sigma^*) = 1 \wedge (\mu^*, \sigma^*) \notin \mathcal{Q}]$$

Definition 3.5 (EUF-KOA Security). A digital signature scheme DS is Existentially Unforgeable under Key-Only Attack (EUF-KOA) if it is computationally infeasible for an adversary $\mathcal{A}$ to produce a valid forgery given only the public key. Unlike the SUF-CMA model, $\mathcal{A}$ lacks access to a signing oracle $\text{OSign}(sk, \cdot)$, effectively setting the query set $\mathcal{Q} = \emptyset$. The advantage of an adversary $\mathcal{A}$ is defined as:

$$\text{Adv}_{\text{DS}}^{\text{EUF-KOA}}(\mathcal{A}) := \Pr \left[\text{Verify}(pk, \mu^*, \sigma^*) = 1 \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen}(1^\lambda) \\ (\mu^*, \sigma^*) \leftarrow \mathcal{A}(pk) \end{array} \right]$$

The following Theorem 3.1 identifies the SUF-CMA security of ATLAS signature in the quantum random oracle model (QROM). The security of the construction relies mainly on the three lattice problems, MLWR, MSIS, and SelfTargetMSIS. Basically, the MLWR problem ensures security against key recovery, the SelfTargetMSIS assumption provides security against new-message forgery, and the MSIS problem is required for strong unforgeability.

Theorem 3.1 (SUF-CMA SECURITY OF ATLAS). Suppose $\mathcal{QA}$ is a quantum adversary against the ATLAS signature scheme that makes q_H queries to the quantum random oracle $|H\rangle$. Then there exist quantum adversaries $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$ that solve the MLWR problem with parameters $\text{pp}_1 = \{k, l, S\}$, the SelfTargetMSIS assumption with parameters $\text{pp}_2 = \{H, k, l+1, \zeta\}$, and the MSIS problem with parameters $\text{pp}_3 = \{k, l, \zeta'\}$, respectively, such that the following holds.

$$\text{Adv}_{\text{ATLAS}}^{\text{SUF-CMA}}(\mathcal{QA}) \leq \text{Adv}_{\text{MLWR}}^{\text{pp}_1}(\mathcal{B}) + \text{Adv}_{\text{SelfTargetMSIS}}^{\text{pp}_2}(\mathcal{C}) + \text{Adv}_{\text{MSIS}}^{\text{pp}_3}(\mathcal{D}) + 2^{-\tau+1}$$

Let S denote the uniform distribution of the secret in the MLWR problem and τ represent the min-entropy of the underlying identification scheme. The other notations, ζ and ζ' , are defined as follows:

$$\begin{aligned} \zeta &= \max\left\{\gamma_1 - \beta_1, \frac{q}{p}(2\bar{\gamma}_2 + 1 + \kappa \cdot 2^{pk_{\text{drop}}-1})\right\} \leq 4\gamma_2 \\ \zeta_1 &= \max\{2(\gamma_1 - \beta_1), 4\gamma_2 + 2\} \leq 4\gamma_2 + 6\eta \end{aligned}$$

The reduction from SUF-CMA to EUF-KOA security follows the framework of Kiltz, Lyubashevsky, and Schaffner [16], who proved that for *zero-knowledge deterministic* signature schemes, any adversary having quantum access to the random oracle $|H\rangle$ and classical access to a signing oracle that can forge a signature on a new message can be converted into an adversary that forges without any access to the signing oracle. That is, given only the public key. The applicability of this result to ATLAS rests on two properties. First, the signing algorithm is *deterministic* in the sense that the randomness $\mathbf{y}$ is derived via a pseudorandom function of the message and secret key, so the scheme fits the deterministic signing model of [16]. Second, the underlying interactive protocol of ATLAS signing algorithm satisfies *non abort honest verifier zero-knowledge*: by the rejection sampling argument, the output distribution of $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ is statistically independent of the secret $\mathbf{s}_1$, and signatures can therefore be perfectly simulated without

the signing key². Together, these two properties guarantee that signing queries provide the adversary with no information beyond what is already available from the public key and the quantum random oracle, so the signing oracle is redundant. Applying the [16] result directly therefore yields

$$\text{Adv}_{\text{ATLAS}}^{\text{SUF-CMA}}(\mathcal{QA}) \leq \text{Adv}_{\text{ATLAS}}^{\text{EUF-KOA}}(\mathcal{QA}'),$$

where $\mathcal{QA}'$ runs in essentially the same time as $\mathcal{QA}$.

Lemma 3.1 (EUF-KOA SECURITY). *Suppose $\mathcal{QA}$ is a quantum adversary against the EUF-KOA-security of the ATLAS signature scheme that issues q_H queries to the quantum random oracle $|H\rangle$. Then, there exist quantum adversaries $\mathcal{B}$ and $\mathcal{C}$ that solve the MLWR problem with parameters $\text{pp}_1 = \{k, l, S\}$ and the SelfTargetMSIS problem with parameters $\text{pp}_2 = \{H, k, l + 1, \zeta\}$, respectively, such that the following holds.*

$$\text{Adv}_{\text{ATLAS}}^{\text{EUF-KOA}}(\mathcal{QA}) \leq \text{Adv}_{\text{MLWR}}^{\text{pp}_1}(\mathcal{B}) + \text{Adv}_{\text{SelfTargetMSIS}}^{\text{pp}_2}(\mathcal{C})$$

Furthermore, the time complexity is bounded by $\text{Time}(\mathcal{B}) = \text{Time}(\mathcal{C}) = \text{Time}(\mathcal{QA}) + q_H$.

The complete proof of Lemma 3.1 is provided in Appendix A.2.

Table 4: Details of Block-size and the corresponding Core-SVP cost

		128-bit	192-bit	256-bit	512-bit
SIS	Block-size	417	664	909	1999
	Core-SVP (Classical)	121	194	265	584
	Core-SVP (Quantum)	110	176	241	530
SIS (SUF)	Block-size	416	668	868	1853
	Core-SVP (Classical)	121	177	253	541
	Core-SVP (Quantum)	110	195	230	491
Primal	Block-size	407	694	948	1636
	Core-SVP (Classical)	119	202	277	574
	Core-SVP (Quantum)	107	184	251	520
Dual	Block-size	406	691	944	1635
	Core-SVP (Classical)	118	202	276	478
	Core-SVP (Quantum)	107	183	250	432

3.3 Algebraic Security

Following the methodology of Dilithium, we derive the security parameters with minor adaptations for the MLWR [4, 22] problem. Since no known attacks currently exploit the module structure, we analyze the MLWR and MSIS problems as their standard counterparts, LWR and SIS. These problems are functionally identical to the definitions of MLWR and MSIS, with the ring $\mathcal{R}_q$ replaced by the scalar ring $\mathbb{Z}_q$.

²Here we sketch the reduction idea. For more details about the naHVZK proof, we refer to [16]

Security of MLWR. An instance of $\text{MLWR}_{q,p,l,k}$ can be viewed as an LWR instance with dimensions nl and nk . Specifically, $\text{MLWR}_{q,p,l,k}$ is equivalent to finding $\text{vec}(\mathbf{s}) \in \mathbb{Z}^{nl} \times \mathbb{Z}^{nk}$ from $(\text{rot}(\mathbf{A}), \text{vec}(\mathbf{t}))$, where $\text{vec}(\cdot)$ maps a vector of $\mathcal{R}_q$ to a vector of its coefficients. The matrix $\text{rot}(\mathbf{A}) \in \mathbb{Z}_q^{nk \times nl}$ is constructed by replacing each entry $a_{ij} \in \mathcal{R}$ with an $n \times n$ matrix whose z -th column is $\text{vec}(x^{z-1} \cdot a_{ij})$.

Given an LWR instance $(\mathbf{A}, \mathbf{t} := \lfloor \frac{p}{q} \mathbf{A} \mathbf{s} \rfloor)$, we convert it to an LWE instance $(\mathbf{A}, \frac{q}{p} \mathbf{t} = \mathbf{A} \mathbf{s} + \frac{q}{p} \boldsymbol{\xi})$. Here, the vector $\boldsymbol{\xi} := \lfloor \frac{p}{q} \mathbf{A} \mathbf{s} \rfloor - \frac{p}{q} \mathbf{A} \mathbf{s}$ represents the rounding error vector, which is uniformly distributed over $(-\frac{1}{2}, \frac{1}{2})$. The noise variance of the resulting LWE sample is $\text{Var} = \frac{q^2}{12p^2}$. We estimate the concrete hardness, represented by the BKZ block size b , using the `lwe-estimator` with parameters 256 l , q , and var .

Security of SelfTargetMSIS. In the security proof, we showed that a successful forgery yields a solution to an instance of the SelfTargetMSIS problem. The most effective attack against the SelfTargetMSIS problem involves both compromising the hash function $\mathbf{H}$ and solving the underlying relation. Since $\mathbf{H}'$ is modelled as a cryptographic hash function, an adversary must first find a value $\mathbf{w}$ such that $\mathbf{H}'(\mu \parallel \mathbf{w}) = c$. For the resulting challenge c , the adversary must then produce a valid pair $(\mathbf{z}, \mathbf{u}')$ satisfying $\mathbf{w} = \mathbf{A} \mathbf{z} - c \frac{q}{p} \mathbf{t} + \frac{q}{p} \mathbf{u}'$. Rearranging the above equation, we obtain

$$\mathbf{A} \mathbf{z} + \frac{q}{p} \mathbf{u}' = \mathbf{t}', \text{ where } \mathbf{t}' := \mathbf{w} + c \frac{q}{p} \mathbf{t}.$$

Therefore, forging a signature reduces to finding vectors $\mathbf{z}$ and $\mathbf{u}'$ that satisfy the above relation while respecting the verification bounds $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$ and $\|\mathbf{u}'\|_\infty < 4 \frac{p}{q} \gamma_2$. Consequently, the concrete security of ATLAS can be estimated through the hardness of the corresponding $\text{MSIS}_{k,l+1,\zeta}$ instance associated with the matrix $[\mathbf{A} \mid \mathbf{t}']$, where the bound ζ is determined by the admissible norms of $\mathbf{z}$ and $\mathbf{u}'$.

Security of MSIS. The above relation naturally gives rise to an instance $\text{MSIS}_{k,l+1,\zeta}$ associated with the augmented matrix $[\mathbf{A} \mid \mathbf{t}']$. To estimate its concrete hardness, we convert this module-SIS instance into an equivalent SIS instance over $\mathbb{Z}_q$ using the standard rotation representation. More precisely, the matrix $[\mathbf{A} \mid \mathbf{t}']$ is expanded into its corresponding rotation matrix $\text{rot}([\mathbf{A} \mid \mathbf{t}']) \in \mathbb{Z}_q^{nk \times n(l+1)}$, yielding an SIS instance of dimension $nk \times n(l+1)$. Similarly, the $\text{MSIS}_{k,l,\zeta'}$ instance is represented by the rotation matrix $\text{rot}(\mathbf{A}) \in \mathbb{Z}_q^{nk \times nl}$, which induces an SIS instance of dimension $nk \times nl$.

We evaluate the concrete hardness of these instances using standard lattice-security estimators. The resulting parameters are comparable to those used in Dilithium, as the values of q , k , l , and the corresponding norm bounds remain in a similar range. Consequently, the induced MSIS instances are expected to provide a comparable level of security, and the estimated security closely follows that of Dilithium, with only minor variations due to the choice of modulus and parameter dimensions.

BKZ Reduction. To estimate the concrete hardness of the induced LWE and SIS instances, we consider attacks based on lattice basis reduction. The most effective known algorithm for finding very short non-zero vectors in Euclidean lattices is the Block, Korkine, Zolotarev (BKZ) algorithm, originally proposed by Schnorr and Euchner [26]. BKZ forms the basis of the best known attacks against SIS, LWE, and related lattice problems. The algorithm operates by repeatedly calling an SVP solver in dimension b , and the overall hardness of the attack therefore scales with the cost of solving SVP in that dimension. Since this cost grows exponentially in b , a sufficiently large block-size renders any BKZ-based attack against ATLAS infeasible. Table 5 summarises the asymptotic complexities of the best known SVP solvers under three computation models.

The paranoid estimate does not correspond to any known algorithm; rather, it represents a conjectured

Setting	Asymptotic Complexity
Classical [5]	$\approx 2^{0.292b}$
Quantum [21]	$\approx 2^{0.265b}$
Paranoid [13]	$\approx 2^{0.2075b}$

Table 5: Asymptotic cost of the best known SVP solvers in dimension b .

lower bound. The best known SVP solvers proceed by covering the b -dimensional unit sphere with cones of centre-to-edge angle $\pi/3$, which requires $\approx 2^{0.2075b}$ cones. Reducing the complexity below $2^{0.2075b}$ would therefore require a fundamental theoretical breakthrough in the geometry of numbers. We refer to the Dilithium specifications document for details regarding the paranoid estimate [13].

Primal and Dual Attack. We describe two attacks based on the BKZ algorithm, known as the primal and dual attacks. The *primal attack* targets an LWE instance $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_q^{\bar{m} \times \bar{n}} \times \mathbb{Z}_q^{\bar{m}}$ with $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, and the norm of the vector $\mathbf{u} = (\mathbf{e}, \mathbf{s}, 1)$ is ϵ . The embedding lattice has dimension $(\bar{m} + \bar{n} + 1)$, and the target vector $\mathbf{u}$ is short relative to the lattice determinant. To solve $\text{LWE}_{\bar{m}, \bar{n}, \epsilon}$, we set the size of the BKZ block b at the smallest integer ≥ 50 that satisfies $\epsilon\sqrt{b} \leq \delta^{2b - (\bar{m} + \bar{n} + 1)} \cdot q^{\bar{m}/(\bar{m} + \bar{n} + 1)}$, where $\delta = ((\pi b)^{1/b} \cdot b / (2\pi e))^{1/(2(b-1))}$ is the root Hermite factor of BKZ- b . The attack succeeds as soon as the reduced basis is short enough to reach the target vector, and the optimal $\bar{m}$ is chosen by minimising the classical gate complexity $2^{0.292b}$ over admissible dimensions.

The *dual attack* searches for a short vector $\mathbf{v}$ satisfying $\mathbf{v}^\top \mathbf{A} \equiv \mathbf{0} \pmod{q}$, and exploits it to distinguish $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ from uniform by computing $\mathbf{v}^\top \mathbf{b} = \mathbf{v}^\top \mathbf{e} \pmod{q}$. When $\|\mathbf{v}\|$ is small, the inner product $\mathbf{v}^\top \mathbf{e}$ is concentrated near zero modulo q , yielding a statistical distinguishing advantage. To solve $\text{LWE}_{\bar{m}, \bar{n}, \epsilon}$, we set the BKZ block size b to the smallest integer ≥ 50 such that $-2\pi^2 \tau(b)^2 \geq \ln(2^{-0.2075b/2})$, where $\tau(b) := \delta^{\bar{m} + \bar{n} - 1} \cdot q^{\bar{n}/(\bar{m} + \bar{n})} \cdot \epsilon/q$. The condition ensures that the distinguishing advantage is large enough to succeed with non-negligible probability, and the classical gate complexity is estimated as $2^{0.292b}$.

3.3.1 Calculating the practical security for ATLAS

Calculating the practice security for a parameter set follows the calculation in [3, 5.2.1]. We provide the calculation for ATLAS-128, the dimension of the lattice is 991 and the block size for the Primal attack is 407 as given in Table 4, then the dimension for free is $d4f(407) = 36.925 \approx 37$ and the effective block size that we use to calculate the gate count is $407 - 37 = 370$. Cost of SVP for block size 370 is $2^{129.96}$, thus the total cost of the attack comes to $(991 - 407) * C^2 * 2^{129.96} \approx 2^{144.04}$.

Table 6: Gate cost of the given parameter sets

Bit Security	Gate Cost ($\log_2$)
ATLAS-128	144.04
ATLAS-192	224.34
ATLAS-256	295.58
ATLAS-512	582.18

3.4 Security Beyond Unforgeability

Beyond standard unforgeability, ATLAS satisfies the *Beyond UnForgeability Features* (BUFF) security properties introduced by Cremers, Düzl , Fiedler, Fischlin, and Janson [11]. The BUFF framework captures three security properties that go beyond EUF-CMA: *exclusive ownership* (M-S-UEO), which ensures that a signature verifies under only one public key and prevents key substitution attacks; *message-bound signatures* (MBS), which ensures that a signature is bound to a unique message; and *non-resignability* (NR), which ensures that an adversary cannot produce a valid signature under a fresh key for an unknown message. The absence of these properties has been shown to enable real-world protocol attacks, including certificate hijacking attacks on early versions of the ACME protocol and path integrity attacks on the DRKey/OPT routing protocol [11].

The ATLAS signature scheme satisfies all three BUFF properties — M-S-UEO, MBS, and NR — by a direct application of Theorem 4.5 of Cremers et al. [11]. That theorem requires two structural conditions on the signature scheme: first, the signature must contain an explicitly verified hash digest of the message and the public key; second, the underlying hash function must be collision resistant and Φ -non-malleable for the class $\Phi = \{\phi_{pk'} \mid \phi_{pk'}(pk, M) = (pk', M)\}$. We verify both conditions for ATLAS below.

Structural condition. The message digest is computed as $\mu = \text{KDF-SM3}(\text{KDF-SM3}(\rho \parallel \mathbf{t}_1) \parallel M)$, where $(\rho, \mathbf{t}_1)$ constitutes the public key and M is the message being signed (see Algorithm 2). The challenge polynomial is then derived as $c \leftarrow H(\mu \parallel \mathbf{w}_1)$, and the signature output is $\sigma = (\mathbf{z}, \mathbf{h}, c)$. The verification algorithm recomputes μ from the public key and message, recomputes $\mathbf{w}_1$ from $(\mathbf{z}, \mathbf{h})$, and accepts only if the recomputed challenge matches c . Therefore, c is an explicitly verified digest of both the public key $(\rho, \mathbf{t}_1)$ and the message M , and it is included in σ . This is precisely the BUFF structure of [11], and carries no additional size overhead since c is already a required component of the signature.

Hash function conditions. For ATLAS, all symmetric operations in ATLAS are instantiated via KDF-SM3 [27]. Collision resistance of KDF-SM3 is a standard assumption following from the collision resistance of the underlying SM3 compression function [27]. For Φ -non-malleability, we follow the argument of [11] for iterated hash functions: since KDF-SM3 processes its input via an iterated construction with an idealized round function, any adversary wishing to produce a valid challenge c' for a modified public key pk' and an unknown message M must query KDF-SM3 on input $(\text{KDF-SM3}(\rho' \parallel \mathbf{t}'_1), M)$. Since M has super-logarithmic min-entropy conditioned on the adversary’s view, this query is infeasible without knowing M , establishing Φ -non-malleability.

Derived guarantees. With both conditions verified, Theorem 4.5 of [11] applies directly. M-S-UEO follows from collision resistance of KDF-SM3: any signature σ verifying under two distinct public keys $pk_1 \neq pk_2$ would yield a collision in KDF-SM3. MBS follows by the same argument: a signature verifying for two distinct messages $M_1 \neq M_2$ under the same public key would yield a collision in KDF-SM3. NR follows from Φ -non-malleability: an adversary in possession of $\sigma = (\mathbf{z}, \mathbf{h}, c)$ for an unknown message M cannot produce a valid signature σ' under a fresh key pk' without computing $\text{KDF-SM3}(\text{KDF-SM3}(\rho' \parallel \mathbf{t}'_1) \parallel M)$, which requires knowledge of M .

3.5 Side-Channel Security

Side-channel attacks attempt to extract secret key material by observing physical leakage such as timing, power consumption, or electromagnetic radiation during cryptographic computations. We discuss the

resilience of **ATLAS** against the three main classes of such attacks.

Timing attacks. A primary source of timing leakage in lattice-based schemes is variable-time operations such as rejection sampling over a prime modulus. **ATLAS** mitigates this by adopting power-of-two moduli ($q = 2^{23}$ or 2^{25} , $p = 2^d$ for $d \in \{18, 19, 20, 22\}$), which allow all modular reductions to be implemented as simple bitmasking operations with no data-dependent branching. The use of **MLWR** rounding instead of discrete error sampling further eliminates the need for variable-time samplers. The implementations of **ATLAS** do not use any conditional jump operation based on secret data, use secret data-dependent branches, or lookup tables to avoid timing attacks [1]. As a result, we believe **ATLAS** provides constant-time implementations across all parameter sets.

Differential analysis. Differential power analysis and electromagnetic analysis exploit statistical correlations between physical leakage and intermediate values that depend on the secret key. In a deployment scenario where an attacker has physical access to the device, masking is the standard countermeasure. The power-of-two modular arithmetic in **ATLAS** is particularly amenable to masking, since arithmetic sharing over $\mathbb{Z}_{2^k}$ is natively compatible with Boolean masking without the modulus mismatch overhead that arises in schemes operating over prime fields [12]. Furthermore, the use of **MLWR** rounding in place of error sampling removes an additional source of masking complexity, since there is no error distribution block to protect. Also, it has been observed [18, 9] that the **MLWR**-based scheme incurs lower performance overhead than **MLWE**-based schemes.

Single-trace attacks. Masked implementations remain vulnerable to single-trace attacks, which process a single power or electromagnetic trace horizontally to recover information about both shares simultaneously. The primary target in **ATLAS** is the polynomial matrix multiplication $\mathbf{Az}$ during signing and verification. Randomised shuffling of the order of arithmetic operations is a standard countermeasure and can be applied on top of a masked implementation at low overhead. We expect that the power-of-two structure of **ATLAS**, which avoids NTT-based multiplication over prime fields, provides implementation advantages similar to those observed for **SABER** [12] in this regard. A detailed implementation-level analysis of single-trace countermeasures for **ATLAS** is left as future work.

4 IMPLEMENTATION OF ATLAS

4.1 Implementation Parameters

- **SEEDBYTES**: The length of the seed used for public matrix generation and randomness expansion.
- **CRHBYTES**: The output length of the collision-resistant hash function.
- **N**: The ring degree corresponding to the theoretical parameter n .
- **QBITS**, **PBITS**: The bit-lengths of the moduli q and p , respectively.
- **Q**, **P**: The modulus values used by the scheme.
- **K**, **L**: The matrix dimensions corresponding to the parameters k and l .
- **D**: The public-key compression parameter corresponding to pk_{drop} .
- **ETA**: The bound used for sampling secret-key coefficients.

- **SETABITS**: The number of bits required to represent coefficients sampled from the secret distribution.
- **GAMMA1_BITS**, **GAMMA2_BITS**: The bit-lengths of the rejection-sampling parameters γ_1 and γ_2 .
- **GAMMA1**, **GAMMA2**: The corresponding values of γ_1 and γ_2 .
- **GAMMA1_BAR_BITS**, **GAMMA2_BAR_BITS**: The bit-lengths of $\bar{\gamma}_1$ and $\bar{\gamma}_2$.
- **GAMMA1_BAR**, **GAMMA2_BAR**: The corresponding values of $\bar{\gamma}_1$ and $\bar{\gamma}_2$.
- **BETA1**, **BETA2**: The rejection-sampling thresholds corresponding to β_1 and β_2 .
- **KAPPA**: The challenge weight parameter κ .
- **OMEGA**: The maximum permitted weight of the hint vector.

Here, we define a rounding operation that maps an element of $\mathbb{Z}_q$ to a compressed p -bit representation by extracting its most significant bits.

ROUND(a). For $a \in \mathbb{Z}$, **ROUND**(a) extracts the higher-order bits of $a \bmod q$, i.e., rounding a to a p -bit representation. We use the following expression to compute the value of it.

$$\text{ROUND}(a) \ ((((a) \ \& \ (Q - 1)) + (1 \ll (QBITS - PBITS - 1))) \gg (QBITS - PBITS))$$

4.2 Algebraic Structures and Ring Arithmetic

4.2.1 Data structure.

The implementation utilizes three primary structures to manage the algebraic objects of the ring $\mathcal{R}_q$:

- **poly**: The fundamental data structure representing a polynomial in the ring $\mathcal{R} = \mathbb{Z}[X]/\langle X^n + 1 \rangle$. It consists of an array **coeffs** of size N .
- **polyvec1**: A polynomial vector of length L , representing elements in $\mathcal{R}_q^L$ such as the random vector **y** or the secret key **s**₁.
- **polyveck**: A polynomial vector of length K , representing elements in $\mathcal{R}_q^K$ such as the public key **t** or the error vector **s**₂.

4.2.2 Arithmetic in $\mathcal{R}_q$

The core ring operations are implemented to support arithmetic in $\mathcal{R}_q = \mathbb{Z}_q[X]/\langle X^n + 1 \rangle$. These functions prioritize constant-time execution and efficient modular reduction.

Additive Subtractions and Scaling. The function **poly_add** and **poly_sub** perform vector addition. Subtraction utilizes a $2q$ offset (**a** + $2 * Q$ - **b**) to ensure results remain positive in an unsigned integer context. The function **poly_shiftl** scales a polynomial by 2^k , effectively performing a coordinate-wise left shift.

Multiplication. The function `pol_mul` performs a multiplication followed by a reduction modulo $X^n + 1$. The product $r(X) = a(X) \cdot b(X)$ is first computed as a linear convolution of length $2N - 1$. Using the identity $X^n \equiv -1 \pmod{X^n + 1}$, the coefficients for $i \geq N$ are folded: $r_{i-N} = (r_{i-N} - r_i) \pmod{q}$.

Norm Verification. The function `poly_chk_norm` evaluates the infinity norm $\|\mathbf{a}\|_\infty = \max |a_i|$. The implementation maps coefficients to the centralized range $\left(-\frac{\text{mod}}{2}, \frac{\text{mod}}{2}\right]$ and uses bitwise operations to compute absolute values in constant time. The function returns 1 if any coefficient greater than the bound B , otherwise it will return 0.

Polynomial randomness with a fixed norm: `poly_uniform_eta`, and `poly_uniform_gamma1m1`. This function samples a polynomial $a \in \mathcal{R}_q$ whose coefficients are uniformly distributed over the integer interval $[-\eta, \eta]$. The sampling is performed using rejection sampling over a pseudorandom byte stream generated via an extendable-output function. Concretely, the input seed `seed` $\in \{0, 1\}^{\text{SEEDBYTES}}$ is concatenated with a one-byte nonce `nonce` to form the input string `seed || nonce`, which is absorbed into `pseudoXOF`. The resulting XOF is then used to generate a stream of pseudorandom bytes. These bytes are processed by a rejection sampling procedure `rej_eta` that maps portions of the byte stream to candidate integers in $[-\eta, \eta]$, accepting only those that fall within the desired range. The procedure continues until all n coefficients of a are assigned. To ensure sufficient output, multiple blocks are squeezed from `pseudoXOF`, and additional blocks are generated if the initial output does not yield enough valid coefficients. The resulting polynomial a is thus distributed uniformly over $\{-\eta, \dots, \eta\}^n$.

The function `poly_uniform_gamma1m1` samples a polynomial $a \in \mathcal{R}_q$ with coefficients uniformly distributed over $[-(\gamma_1 - 1), \gamma_1 - 1]$. It follows the same rejection sampling paradigm as `poly_uniform_eta`, using a pseudorandom byte stream derived from `pseudoXOF(seed || nonce)`, where the nonce is encoded in little-endian form. The stream is processed by `rej_gamma1m1` to obtain valid coefficients, and additional output blocks are generated if necessary until all n coefficients are assigned.

4.3 Polynomial Multiplication

The ATLAS signature scheme is heavily dependent on polynomial arithmetic in the ring $\mathcal{R}_q$ with power-of-two modulus $q = 2^{23}$ and the irreducible polynomials $X^n + 1$, where n is set to 128 for $\lambda = 128 \& 192$, 256 for $\lambda = 256$ and 512 for $\lambda = 512$. While polynomial addition and subtraction involve simple coefficient-wise operations, polynomial multiplication is costly, requiring in the order of $\mathcal{O}(n^2)$ computations. As a result, it is essential to optimise the polynomial multiplication routine for improving the efficiency of ATLAS. Unlike Dilithium, the Number Theoretic Transform (NTT) cannot be used owing to a non-prime q , necessitating the choice of Karatsuba and Toom-Cook polynomial multiplication methods. The Karatsuba multiplication follows a top-down recursive approach: an n degree polynomial multiplication is recursively split into three $n/2$ degree polynomial multiplications. After several such splittings, when the polynomial degree reaches a particular threshold, a quadratic-complexity polynomial multiplication algorithm, such as the Schoolbook method, is used to compute the product of the smaller-degree polynomials. For example, when $n = 128$ and the chosen threshold is 8, we need to perform this splitting 4 times, and the Schoolbook multiplication routine will be called $3^4 = 81$ times. This method offers an asymptotic complexity of $\mathcal{O}(n^{\log_2 3})$ and its pseudo-code is listed in Algorithm 16 in Appendix B.

This can be further improved by using the Toom-Cook polynomial multiplication algorithm. This algorithm is a generalisation of the Karatsuba method and can be used to split n -degree polynomial multiplication into seven $n/4$ -degree polynomial multiplications. This is known as the *four-way Toom-*

Cook multiplication, denoted as Toom-4 henceforth, and has been shown in Algorithm 19, listed in Appendix B. The smaller degree multiplications can be done by a combination of the Karatsuba and the Schoolbook method described above. For example, if $n = 128$, one iteration of Toom-4 splits the multiplication into seven 32 degree multiplications. Applying Karatsuba in the next recursions with the previously mentioned threshold set to 8, the number of calls for the Schoolbook routine can be reduced to $7 \times 3^2 = 63$. The Toom-4 polynomial multiplication has an asymptotic complexity of $\mathcal{O}(n^{\log_4 7})$, making it better than the Karatsuba method.

The evaluation step in Toom-4 multiplication is done for the points $y = \{0, \pm 1, \pm \frac{1}{2}, 2, \infty\}$, as per [7]. In the interpolation phase, multiplications and divisions by scalar constants need to be computed. Divisions by odd scalars are performed using their multiplicative inverses modulo 2^{32} . Since inverses do not exist for even divisors with respect to such a power-of-two modulus, division by an even scalar $f = 2^m \cdot a$ is computed in two steps: first, we multiply by the inverse of the odd factor a , then we compute the division by right-shifting by m positions. For example, division by $24 = 2^3 \cdot 3$ is performed by first multiplying by the multiplicative inverse of 3, then right-shifting by 3 positions. For the Toom-4 algorithm, the maximum power-of-two factor for performing such a two-step division by an even constant is 8, which can lead to a loss of 3 bits of precision. As a result, we use 32-bit coefficients such that the extra bits can incorporate divisions by factors of 2, 4, and 8 during divisions by 18, 60 and 24, respectively.

4.3.1 Striding Toom-Cook and Karatsuba Multiplication

As described above, classical Toom-Cook partitions the input polynomial into contiguous coefficient blocks. The polynomial $A(x) = \sum_{i=0}^{N-1} a_i x^i$ is decomposed as $A(x) = \sum_{i=0}^{k-1} x^{im} A_i(x)$ where $A_i(x) = \sum_{j=0}^{m-1} a_{im+j} x^j$ and $m = \frac{N}{k}$. For example, a classical Toom-4 decomposition gives

$$A_0 = (a_0, \dots, a_{m-1}); A_1 = (a_m, \dots, a_{2m-1}); A_2 = (a_{2m}, \dots, a_{3m-1}); A_3 = (a_{3m}, \dots, a_{4m-1}).$$

The drawback with this approach is that although the final multiplication is performed in the target ring, every recursive base multiplication produces intermediate results whose degree is twice that of the sub-polynomials, requiring temporary storage for the unreduced products. Consequently, repeated recursive decomposition increases memory usage and coefficient rearrangement across successive recursion levels. To address this, we propose a striding formulation based on [6] which groups coefficients according to their residue modulo the split factor,

$$A(x) = \sum_{i=0}^{k-1} x^i A_i(x^k), \text{ where } A_i(y) = \sum_{j=0}^{m-1} a_{kj+i} y^j$$

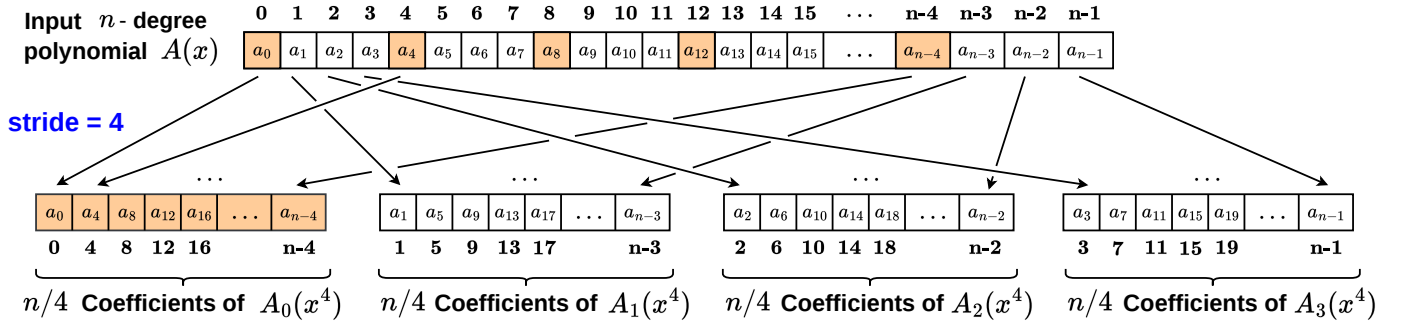
Thus, a striding Toom-4 decomposition is given by

$$A_0 = (a_0, a_4, a_8, \dots); A_1 = (a_1, a_5, a_9, \dots); A_2 = (a_2, a_6, a_{10}, \dots); A_3 = (a_3, a_7, a_{11}, \dots).$$

Similarly, striding Karatsuba decomposition is represented as

$$A(x) = A_e(x^2) + x A_o(x^2), \text{ where } A_e(y) = \sum_j a_{2j} y^j, \text{ and } A_o(y) = \sum_j a_{2j+1} y^j.$$

Difference in Evaluation: The evaluation points and evaluation matrix remain identical to those of the classical Karatsuba and Toom-4 algorithms, as described in Algorithms 17 and 20, respectively.



Classical Toom-4 Decomposition: $A(x) = A_0(x) + x^{n/4} A_1(x) + x^{2n/4} A_2(x) + x^{3n/4} A_3(x)$

Striding Toom-4 Decomposition: $A(x) = A_0(x^4) + x A_1(x^4) + x^2 A_2(x^4) + x^3 A_3(x^4)$

Figure 1: Polynomial decomposition for a striding Toom-4 multiplication subroutine for n -degree input polynomial $A(x)$. Instead of a contiguous memory split, coefficients are accessed at a stride length of 4.

The only difference is the polynomial representation on which the evaluation is performed. Classical decomposition evaluates contiguous coefficient blocks, whereas the striding implementation evaluates residue-class polynomials, as shown in Figure 1. Consequently, the evaluation arithmetic is unchanged, while the coefficient access pattern changes from contiguous to stride- k accesses. This observation applies equally to striding Karatsuba ($k = 2$) and striding Toom- k multiplication.

Difference in Interpolation: In the conventional Toom-Cook formulation, the interpolated polynomial is reconstructed as $C(x) = \sum_{i=0}^{2k-2} X^{im} C_i(X)$, where each interpolation polynomial $C_i(X)$ occupies a distinct coefficient interval. Consequently, the reconstructed coefficient blocks are disjoint and can be written independently, as shown in Algorithms 18 and 21. In the striding formulation, the reconstructed polynomial is instead expressed as $C(x) = \sum_{i=0}^{2k-2} X^i C_i(X^k)$. Since successive interpolation polynomials are separated by only one power of X , the higher-degree coefficients of $C_i(X^k)$ overlap with the lower-degree coefficients of $C_{i+1}(X^k)$. Therefore, interpolation is performed as an overlap-add operation, where the overlapping coefficients are accumulated before the final negacyclic wraparound correction. This property holds for both striding Karatsuba ($k = 2$) and striding Toom- k , differing only in the number of interpolation polynomials involved, and is illustrated in Algorithms 8 and 9, respectively.

Algorithm 8 Striding Karatsuba Interpolation**Input:** Interpolated coefficients w_0, w_1, w_2 obtained from classical Karatsuba interpolation**Output:** Result polynomial res

```

1: Perform classical Karatsuba interpolation to obtain  $[w_0, w_1, w_2]$  as per Algorithm 18
2: for  $i = 0$  to  $n/2 - 1$  do
    // Reconstruct coefficients for current stride
3:    $res[2i] \leftarrow res[2i] + r_0$ 
4:    $res[2i + 1] \leftarrow res[2i + 1] + r_1 - r_0 - r_2$ 
5:   if  $i = n/2 - 1$  then
    // Final negacyclic wraparound
6:      $res[0] \leftarrow res[0] - r_2$ 
7:   else
    // Accumulate overlap into the next stride
8:      $res[2(i + 1)] \leftarrow res[2(i + 1)] + r_2$ 
9:   end if
10: end for

```

Algorithm 9 Striding Toom-4 Interpolation**Input:** Interpolated coefficients $w_0, \dots, w_6$ from standard Toom-4 interpolation**Output:** Result polynomial res

```

1: Perform classical Toom-4 interpolation to obtain  $[w_0, w_1, \dots, w_6]$  as per Algorithm 21
2:  $prev_0 \leftarrow 0, prev_1 \leftarrow 0, prev_2 \leftarrow 0$ 
   // Striding writeback using overlapping additions
3: for  $i = 0$  to  $n/4 - 1$  do
4:    $res[4i + 3] \leftarrow r_3$ 
5:   if  $i = 0$  then
6:      $res[4i] \leftarrow r_6$ 
7:      $res[4i + 1] \leftarrow r_5$ 
8:      $res[4i + 2] \leftarrow r_4$ 
9:   else
10:     $res[4i] \leftarrow r_6 + prev_2$ 
11:     $res[4i + 1] \leftarrow r_5 + prev_1$ 
12:     $res[4i + 2] \leftarrow r_4 + prev_0$ 
13:  end if
14:   $prev_0 \leftarrow r_0$ 
15:   $prev_1 \leftarrow r_1$ 
16:   $prev_2 \leftarrow r_2$ 
17: end for
   // Final negacyclic wraparound
18:  $res[0] \leftarrow res[0] - prev_2$ 
19:  $res[1] \leftarrow res[1] - prev_1$ 
20:  $res[2] \leftarrow res[2] - prev_0$ 

```

4.3.2 Design Space Exploration and Multiplication Strategies

We employ a hierarchical decomposition framework to evaluate multiple combinations of polynomial multiplication for degree-128, degree-256, and degree-512 polynomials using both classical and striding implementations. At each recursion level, a multiplication strategy, such as Toom-4 or Karatsuba, is selected, while the schoolbook method serves as the base multiplication. The overall algorithm, shown in Algorithm 10, is parameterized by a strategy sequence $\mathcal{S}$, where each element specifies the multiplication method applied at a given recursion depth. The recursion terminates at a predefined base threshold, where degree-8 or degree-16 schoolbook multiplication is performed. At each level, the selected strategy determines the corresponding polynomial decomposition, evaluation, and interpolation procedures, while recursive calls compute the pointwise products.

Based on the final strategy selection, the ATLAS-128 and ATLAS-192 parameter sets employ two levels of strided Toom-4 followed by degree-8 schoolbook multiplication, as summarized in Algorithm 11. For the ATLAS-256 parameter set, the selected strategy consists of two levels of striding Toom-4, one level of striding Karatsuba, and degree-8 schoolbook multiplication, as shown in Algorithm 12. Finally, the ATLAS-512 parameter set employs three levels of striding Toom-4 followed by degree-8 schoolbook multiplication, as presented in Algorithm 13. The rationale for selecting these hybrid multiplication strategies is established through the performance evaluation presented in Section 5.1.1.

Algorithm 10 `polymul_recurse`

Input: Polynomials $A(x), B(x)$ of degree n , Strategy list $\mathcal{S} = [S_1, S_2, \dots, S_k]$, Base threshold n_{base}

Output: $C(x) = A(x) * B(x)$

```

1: if  $\mathcal{S}$  is empty or  $n \leq n_{\text{base}}$  then
2:   return Schoolbook $_{n_{\text{base}} \times n_{\text{base}}}(A, B)$ 
3: end if
4:  $S \leftarrow$  first element of  $\mathcal{S}$ 
5:  $\mathcal{S}' \leftarrow$  remaining strategy list
   // Splitting based on strategy  $S$ 
6: if  $S = \text{Toom-4}$  then
7:   Set  $n' = n/4$  and  $y = x^{n'}$ 
8:   Split  $A(x), B(x)$  into 4 parts:  $A(y) = A_3y^3 + A_2y^2 + A_1y + A_0$ ,  $B(y) = B_3y^3 + B_2y^2 + B_1y + B_0$ 
9:   Set evaluation points  $y_i = \{0, \pm 1, \pm \frac{1}{2}, 2, \infty\}$ 
10: else if  $S = \text{Karatsuba}$  then
11:   Set  $n' = n/2$  and  $y = x^{n'}$ 
12:   Split  $A(x), B(x)$  into 2 parts:  $A(y) = A_1y + A_0$ ,  $B(y) = B_1y + B_0$ 
13:   Set evaluation points  $y_i = \{0, 1, \infty\}$ 
14: end if
15: for each evaluation point  $y_i$  do
16:    $w_i \leftarrow \text{polymul\_recurse}(A(y_i), B(y_i), n', \mathcal{S}')$  ▷ Recursive Polynomial Multiplications
17: end for
18: Recover coefficients from  $\{w_i\}$  based on interpolation formula for  $S$  ▷ Interpolation
19: return recomposed polynomial  $C(x)$ 

```

Algorithm 11 Pseudocode for the final degree-128 polynomial multiplication strategy (2-level Striding Toom-Cook 4-way + 8×8 Schoolbook Multiplication)

Input: Two polynomials $A(x)$ and $B(x)$ of length $n = 128$

Output: $C(x) = A(x) * B(x)$

```

// Evaluation points for Level-1 Toom-4:  $\{0, \pm 1, \pm \frac{1}{2}, 2, \infty\}$ 
1:  $[A^{(0)}(x), \dots, A^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $A(x)$  ▷ Level-1 Evaluation for  $A$ 
2:  $[B^{(0)}(x), \dots, B^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $B(x)$  ▷ Level-1 Evaluation for  $B$ 
3: for  $i = 0$  to 6 do
4:    $[A_i^{(0)}(x), \dots, A_i^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $A^{(i)}(x)$  ▷ Level-2 Evaluation for  $A$ 
5:    $[B_i^{(0)}(x), \dots, B_i^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $B^{(i)}(x)$  ▷ Level-2 Evaluation for  $B$ 
6:   for  $j = 0$  to 6 do
7:      $r_{i,j}(x) \leftarrow$  Schoolbook $_{8 \times 8}(A_i^{(j)}(x), B_i^{(j)}(x))$  ▷ Base multiplication
8:   end for
9:    $r_i(x) \leftarrow$  Striding Toom-4 interpolation on  $[r_{i,0}(x), \dots, r_{i,6}(x)]$  ▷ Level-2 Interpolation
10: end for
11:  $C(x) \leftarrow$  Striding Toom-4 interpolation on  $[r_0(x), \dots, r_6(x)]$  ▷ Level-1 Interpolation
12: return  $C(x)$ 

```

Algorithm 12 Pseudocode for the final degree-256 polynomial multiplication strategy (2-level Striding Toom-Cook 4-way + 1-Level Striding Karatsuba + 8×8 Schoolbook Multiplication)

Input: Two polynomials $A(x)$ and $B(x)$ of length $n = 256$

Output: $C(x) = A(x) * B(x)$

```

// Evaluation points for Level-1 Toom-4:  $\{0, \pm 1, \pm \frac{1}{2}, 2, \infty\}$ 
1:  $[A^{(0)}(x), \dots, A^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $A(x)$  ▷ Level-1 Evaluation for  $A$ 
2:  $[B^{(0)}(x), \dots, B^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $B(x)$  ▷ Level-1 Evaluation for  $B$ 
3: for  $i = 0$  to 6 do
4:    $[A_i^{(0)}(x), \dots, A_i^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $A^{(i)}(x)$  ▷ Level-2 Evaluation for  $A$ 
5:    $[B_i^{(0)}(x), \dots, B_i^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $B^{(i)}(x)$  ▷ Level-2 Evaluation for  $B$ 
6:   for  $j = 0$  to 6 do
7:     // Evaluation points for Karatsuba:  $\{0, 1, \infty\}$ 
8:      $[A_{i,j}^{(0)}(x), A_{i,j}^{(1)}(x), A_{i,j}^{(2)}(x)] \leftarrow$  Striding Karatsuba evaluation of  $A_i^{(j)}(x)$ 
9:      $[B_{i,j}^{(0)}(x), B_{i,j}^{(1)}(x), B_{i,j}^{(2)}(x)] \leftarrow$  Striding Karatsuba evaluation of  $B_i^{(j)}(x)$ 
10:    for  $k = 0$  to 2 do
11:       $t_{i,j,k}(x) \leftarrow$  Schoolbook $_{8 \times 8}(A_{i,j}^{(k)}(x), B_{i,j}^{(k)}(x))$  ▷ Base Multiplication
12:    end for
13:     $r_{i,j}(x) \leftarrow$  Striding Karatsuba interpolation on  $[t_{i,j,0}(x), t_{i,j,1}(x), t_{i,j,2}(x)]$ 
14:  end for
15:   $r_i(x) \leftarrow$  Striding Toom-4 interpolation on  $[r_{i,0}(x), \dots, r_{i,6}(x)]$  ▷ Level-2 Interpolation
16: end for
17:  $C(x) \leftarrow$  Striding Toom-4 interpolation on  $[r_0(x), \dots, r_6(x)]$  ▷ Level-1 Interpolation
18: return  $C(x)$ 

```

Algorithm 13 Pseudocode for the final degree-512 polynomial multiplication strategy (3-level Striding Toom-Cook 4-way + 8×8 Schoolbook Multiplication)

Input: Two polynomials $A(x)$ and $B(x)$ of length $n = 512$

Output: $C(x) = A(x) * B(x)$

```

// Evaluation points for Toom-4:  $\{0, \pm 1, \pm \frac{1}{2}, 2, \infty\}$ 
1:  $[A^{(0)}(x), \dots, A^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $A(x)$  ▷ Level-1 Evaluation for  $A$ 
2:  $[B^{(0)}(x), \dots, B^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $B(x)$  ▷ Level-1 Evaluation for  $B$ 
3: for  $i = 0$  to 6 do
4:    $[A_i^{(0)}(x), \dots, A_i^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $A^{(i)}(x)$  ▷ Level-2 Evaluation for  $A$ 
5:    $[B_i^{(0)}(x), \dots, B_i^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $B^{(i)}(x)$  ▷ Level-2 Evaluation for  $B$ 
6:   for  $j = 0$  to 6 do
7:      $[A_{ij}^{(0)}(x), \dots, A_{ij}^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $A_i^{(j)}(x)$  ▷ Level-3 Evaluation for  $A$ 
8:      $[B_{ij}^{(0)}(x), \dots, B_{ij}^{(6)}(x)] \leftarrow$  Striding Toom-4 evaluation of  $B_i^{(j)}(x)$  ▷ Level-3 Evaluation for  $B$ 
9:     for  $k = 0$  to 6 do
10:       $r_{ijk}(x) \leftarrow$  Schoolbook $_{8 \times 8}(A_{ij}^{(k)}(x), B_{ij}^{(k)}(x))$  ▷ Base multiplication
11:    end for
12:     $r_{ij}(x) \leftarrow$  Striding Toom-4 interpolation on  $[r_{ij0}(x), \dots, r_{ij6}(x)]$  ▷ Level-3 Interpolation
13:  end for
14:   $r_i(x) \leftarrow$  Striding Toom-4 interpolation on  $[r_{i0}(x), \dots, r_{i6}(x)]$  ▷ Level-2 Interpolation
15: end for
16:  $C(x) \leftarrow$  Striding Toom-4 interpolation on  $[r_0(x), \dots, r_6(x)]$  ▷ Level-1 Interpolation
17: return  $C(x)$ 

```

4.4 Packing and Unpacking

4.4.1 Packing and Unpacking of Vectors

This subsection describes the polynomial packing and unpacking routines used in ATLAS. We present the serialization procedures for $\mathbf{t}_1$, $\mathbf{t}_0$, $\mathbf{z}$, and $\mathbf{w}_1$, namely `polyt1_pack()`, `polyt1_unpack()`, `polyt0_pack()`, `polyt0_unpack()`, `polyz_pack()`, `polyz_unpack()`, and `polyw1_pack()`. These routines are subsequently used in the construction of the public-key, secret-key, and signature encodings.

polyt1_pack() / polyt1_unpack(): *Packing and unpacking of $\mathbf{t}_1$.* The polynomial $\mathbf{t}_1$ forms part of the public key and therefore must be stored in a compact format. Since each coefficient of $\mathbf{t}_1$ fits within 9 bits, we employ a bit-packing procedure that encodes eight coefficients into nine consecutive bytes. More precisely, the packing algorithm concatenates the binary representations of eight 9-bit coefficients and stores the resulting 72 bits without any padding. Consequently, a polynomial of degree n requires only $\frac{9n}{8}$ bytes of storage.

The corresponding unpacking procedure reverses this process by extracting the individual 9-bit coefficients from the packed byte stream and reconstructing the original polynomial. Since the transformation is lossless, the unpacking algorithm exactly recovers the coefficients of $\mathbf{t}_1$. These routines are used during key generation, public-key serialization, and public-key parsing.

polyt0_pack() / polyt0_unpack(): *Packing and unpacking of $\mathbf{t}_0$.* The polynomial $\mathbf{t}_0$ forms the low-

order component of the public-key vector and is stored in compressed form to reduce the secret-key size. By construction, each coefficient of $\mathbf{t}_0$ lies in a bounded interval determined by the parameter pk_{drop} and can therefore be represented using pk_{drop} bits. The packing procedure first maps the signed coefficients to non-negative representatives and then serializes their binary representations into a contiguous byte stream without introducing any padding. Consequently, a polynomial of degree n requires $\frac{n \cdot pk_{drop}}{8}$ bytes of storage.

The unpacking procedure reverses this process by extracting the coefficient representations from the serialized byte stream and recovering the corresponding signed values. Since no information is discarded during serialization, the transformation is lossless and the original polynomial $\mathbf{t}_0$ is reconstructed exactly. These routines are used during secret-key serialization and parsing.

polyz_pack() / polyz_unpack(): *Packing and unpacking of $\mathbf{z}$.* The vector $\mathbf{z}$ forms the main component of the signature and therefore requires an efficient serialized representation. By construction, each coefficient of $\mathbf{z}$ lies in the interval $[-(\gamma_1 - 1), \gamma_1 - 1]$. The packing procedure first maps these signed coefficients to non-negative representatives and then encodes them into a compact byte stream using the minimum number of bits required to represent values in the range $[0, 2\gamma_1 - 2]$. Consequently, the storage cost of a polynomial depends on the choice of γ_1 and scales linearly with the ring degree n .

The unpacking procedure performs the inverse transformation by recovering the coefficient representations from the serialized byte stream and mapping them back to their original signed values. Since the serialization process is lossless, the original polynomial is reconstructed exactly. These routines are used during signature generation, signature serialization, and signature verification.

polyw1_pack(): *Packing of $\mathbf{w}_1$.* The polynomial $\mathbf{w}_1$ corresponds to the high-order component obtained from the decomposition procedure and is used during challenge generation. By construction, each coefficient of $\mathbf{w}_1$ belongs to a small range determined by the decomposition parameter $\bar{\gamma}_2$. For the selected ATLAS parameter sets, every coefficient can be represented using 4 bits. The packing procedure therefore stores two coefficients in a single byte by placing one coefficient in the lower nibble and the other in the upper nibble. Consequently, a polynomial of degree n requires only $\frac{n}{2}$ bytes of storage. Since $\mathbf{w}_1$ is transmitted as part of the signature transcript, this compact representation contributes directly to reducing the overall signature size.

4.4.2 Algorithms for Packing and Unpacking: Keys and Signature

We describe the bit-packing and unpacking keys and signatures into compact byte representations.

pack_pk / unpack_pk. These functions serialize and deserialize the public key $pk = (\rho, \mathbf{t}_1)$. The packing procedure stores ρ as a byte string, followed by the packed representation of each polynomial in $\mathbf{t}_1$ using **polyt1_pack** in a fixed order. The unpacking procedure reverses this process by first extracting ρ and then reconstructing each component of $\mathbf{t}_1$ via **polyt1_unpack**.

pack_sk / unpack_sk. These functions serialize and deserialize the secret key $sk = (\rho, \mathbf{key}, tr, \mathbf{s}_1, \mathbf{t}_0)$. The packing procedure concatenates the byte strings ρ , **key**, and **tr**, followed by the packed representations of the vectors $\mathbf{s}_1$ and $\mathbf{t}_0$ using **polyeta_pack** and **polyt0_pack**, respectively. The unpacking procedure parses the byte array in the same order and reconstructs each component using the corresponding unpacking routines.

pack_sig / unpack_sig. These functions serialize and deserialize the signature $\sigma = (\mathbf{z}, \mathbf{h}, c)$. The vector

$\mathbf{z}$ is encoded using `polyz_pack`. The hint vector $\mathbf{h}$ is stored in a sparse format by recording the indices of non-zero coefficients, along with boundary markers for each component. The challenge polynomial c is encoded by storing the support positions in a bitmask and the corresponding signs in a compact form. The unpacking procedure reverses these steps to reconstruct $\mathbf{z}$, $\mathbf{h}$, and c deterministically.

4.5 Symmetric Primitives

We utilize a custom eXtensible Output Function (XOF) for public matrix expansion, secret material sampling, and deterministic challenge generation. We denote by `pseudoXOF`(M, ℓ) the function that takes an input message M and produces an output of ℓ bits. This primitive is constructed using the KDF-SM3 framework [27] to provide the pseudorandom stream required by the Fiat-Shamir with aborts paradigm. As specified in our reference implementation, all algebraic objects including the public matrix $\mathbf{A}$, the masking vectors $\mathbf{y}$, and the challenge polynomial c are derived deterministically from their respective seeds or message digests via `pseudoXOF`.

4.5.1 Hashing to the Challenge Set

Signature generation requires a challenge polynomial that is sparse and has coefficients in $\{-1, 0, 1\}$. We define $\mathbf{B}_\kappa$ as the set of all polynomials in $\mathcal{R}$ with exactly κ non-zero coefficients, each equal to ± 1 . The size of this set is $|\mathbf{B}_\kappa| = 2^\kappa \binom{n}{\kappa}$. In **ATLAS** the challenge polynomial is produced by a deterministic two-stage hash-to-challenge procedure. In the first stage, an arbitrary-length input string is processed by SM3 [27], a second-preimage resistant hash function, to produce a fixed-length digest. This digest serves as a compact representation of the challenge and acts as the seed for the second stage. In the second stage, `pseudoXOF` expands the digest to derive an element of $\mathbf{B}_\kappa$, determining both the positions of the κ non-zero coefficients and their signs. The resulting polynomial belongs to $\mathbf{B}_\kappa$ by construction.

This two-stage approach offers a practical implementation advantage. Rather than storing or transmitting the full challenge polynomial, it suffices to retain the fixed-length digest from the first stage. The challenge polynomial can then be reconstructed at any point by re-running the `pseudoXOF` expansion. The full procedure is given in Algorithm 14.

Algorithm 14 `SampleInBall`(ρ)

Input: Seed ρ

Output: Challenge polynomial $\mathbf{c} \in \mathbf{B}_\kappa$

```

1: Initialize  $\mathbf{c} = (c_0, c_1, \dots, c_{n-1}) \leftarrow (0, \dots, 0)$ 
   // Insert  $\kappa$  non-zero coefficients at random positions
2: for  $i = n - \kappa$  to  $n - 1$  do
3:   Sample  $j \xleftarrow{\$} \{0, 1, \dots, i\}$ 
4:   Sample  $s \xleftarrow{\$} \{0, 1\}$ 
5:    $c_i \leftarrow c_j$ 
6:    $c_j \leftarrow (-1)^s$ 
7: end for
8: return  $\mathbf{c}$ 
```

4.6 Sampling Randomness

4.6.1 Generation of the Public Matrix

The public matrix $\mathbf{A} \in \mathcal{R}_q^{k \times l}$ is generated deterministically from a 32-byte seed ρ using the `expand_mat` function. This ensures that $\mathbf{A}$ is publicly reproducible from ρ alone and does not need to be transmitted in its entirety, which significantly reduces the public key size. For each entry $a_{i,j}$ at row $i \in \{0, \dots, k-1\}$ and column $j \in \{0, \dots, l-1\}$, the coefficients are sampled as $a_{i,j} := \text{SampleUniform}(\text{pseudoXOF}(\rho \parallel i \parallel j))$. The sampling uses rejection sampling to map the pseudorandom output of `pseudoXOF` to $\mathbb{Z}_q$. A candidate $v \in \{0, \dots, 2^{23} - 1\}$ is accepted if and only if $v < q$, ensuring that the distribution of each coefficient is statistically indistinguishable from uniform over $\mathbb{Z}_q$.

4.6.2 Sampling Challenges.

This function implements the Fiat–Shamir transform by deterministically sampling a sparse challenge polynomial $c \in \mathcal{R}_q$ from the hash of the message digest μ and the commitment $\mathbf{w}_1$. Concretely, the input μ is concatenated with the packed representation of $\mathbf{w}_1$, and the result is absorbed into `pseudoXOF` to generate a pseudorandom byte stream. The polynomial c is then constructed to have exactly κ non-zero coefficients in $\{-1, 1\}$, with positions selected via rejection sampling without replacement and signs derived from the initial bits of the output stream. All remaining coefficients are set to zero. This deterministic procedure ensures that c is uniformly distributed over the set of sparse polynomials with κ non-zero entries, as required by the Fiat–Shamir with aborts paradigm.

4.7 NGCC API Implementation

The implementation of the NGCC API follows the modular architecture of `ATLAS` and consists of the following three algorithms:

- `sig_keygen`: Outputs a pair $(pk, sk) \in \text{PK_SIZE_PACKED} \times \text{SK_SIZE_PACKED}$ bytes, where pk and sk are compact representations of the public and secret keys, respectively. This algorithm generates a random seed ρ to expand the public matrix $\mathbf{A}$, samples the secret vector $\mathbf{s}_1$, and computes the public key component $\mathbf{t}_1$ before invoking the packing routines.
- `sig_sign`: Takes as input a secret key sk and a message M of length len . It first calls `unpack_sk` to expand the secret key into its components, then executes the signing logic to produce a signature $\sigma = (\mathbf{z}, \mathbf{h}, c)$. It outputs a signed message sm , which is the concatenation of the packed signature and the original message.
- `sig_verify`: Takes as input a signed message sm and a public key pk . It calls `unpack_pk` and `unpack_sig` to retrieve the necessary algebraic objects and then invokes the verification routine. If the signature is valid, it outputs the original message M and returns a success status.

4.8 Known Answer Test Values

The submission includes Known Answer Test (KAT) files containing tuples of secret keys (sk), public keys (pk), signatures (sm), messages(msg), and seeds (seed) for each implementation tier: ATLAS-128, ATLAS-192, ATLAS-256, and ATLAS-512. These validation files are located within the `Test_Vector` folders of the submission package.

5 PERFORMANCE EVALUATION

In this section, the implementation aspects of ATLAS are discussed, and the performance of its C reference implementation is analysed. We remark that the reference implementation included in the submission package, as requested by ICCS. Additionally, all function calls to symmetric primitives are implemented using the `pseudoHash` and `pseudoXOF` primitives provided by ICCS.

5.1 Performance Test

Implementation Environment and Platform Configuration. The performance evaluation of the MLWR signature scheme and the associated polynomial multiplication strategies was conducted on a desktop workstation, a Lenovo ThinkCentre M920t, equipped with an Intel Core i7-9700 processor featuring 8 logical cores and a base clock frequency of 3.00 GHz (with a typical operating frequency of 2.6 GHz during the benchmark tests). The system is supported by 16.0 GiB of RAM and runs on Ubuntu 22.04.5 LTS (64-bit) using the Wayland windowing system. All implementations were written in C/C++ and compiled using the gcc 11.4.0 compiler infrastructure with standard optimization flags (`-O2` for reference implementation and polynomial multiplication strategies; `-O3` for optimized implementation as mentioned in the ICCS requirements [15]) to leverage modern instruction sets. Performance metrics, specifically clock cycles (ticks), were captured using the processor’s hardware performance counters to ensure high-precision measurement across the key generation, signing, and verification phases (see Table 11). Each operation has been executed 9,000 times, and the average, and average execution times are reported.

5.1.1 Performance Comparison of Polynomial Multiplication Strategies

This subsection justifies the hybrid multiplication strategies adopted in Section 4.3.2 through a comparative performance evaluation. The performance of multiple polynomial multiplication strategies is evaluated in terms of latency and memory utilization using both conventional and striding implementations.

The performance comparisons of multiple strategies for $n = 128$, $n = 256$, and $n = 512$ using the conventional Toom-4 and Karatsuba implementations are presented in Tables 7, 8, and 9, respectively. Among the non-striding implementations, the optimal strategies are two levels of Toom-4 followed by degree-8 schoolbook multiplication for $n = 128$, two levels of Toom-4 followed by degree-16 schoolbook multiplication for $n = 256$, and two levels of Toom-4, one level of Karatsuba, and degree-16 schoolbook multiplication for $n = 512$.

The introduction of striding evaluation and interpolation in these implementations alters the relative

performance of the polynomial multiplication strategies, as has been illustrated in Table 10. While the optimal strategy for the degree-128 parameter set remains unchanged, the best-performing configurations for the degree-256 and degree-512 parameter sets differ from their non-striding counterparts. For the ATLAS-256 parameter set, although the non-striding implementation comprising two levels of Toom-4 followed by degree-16 schoolbook multiplication outperforms the remaining non-striding hybrid strategies, the corresponding striding implementation is surpassed by the combination of two levels of Toom-4, one level of Karatsuba, and degree-8 schoolbook multiplication. Similarly, for the highest security parameter set, the striding implementation consisting of three levels of Toom-4 followed by degree-8 schoolbook multiplication outperforms its non-striding counterpart as well as the remaining hybrid strategies. Consequently, the hybrid multiplication strategies described in the Section 4.3.2 are adopted for the final implementation of ATLAS.

Table 7: **Average Performance for Different Non-striding Multiplication Strategies** ($n = 128$)

Multiplication Strategy	Avg CPU Cycles	Avg Time (ns)	Avg Memory (KB)
4Karatsuba + 8Schoolbook	14170.4	4229.3	2496
3Karatsuba + 16Schoolbook	11260.0	3434.3	2496
1Toom-4 + 2Karatsuba + 8Schoolbook	10558.1	3208.2	2496
1Toom-4 + 1Karatsuba + 16Schoolbook	8601.4	2623.8	2496
2Toom-4 + 8Schoolbook	7938.8	2418.8	2496

Table 8: **Average Performance for Different Non-striding Multiplication Strategies** ($n = 256$)

Multiplication Strategy	Avg CPU Cycles	Avg Time (ns)	Avg Memory (KB)
5Karatsuba + 8Schoolbook	42997.1	13010.5	2376
4Karatsuba + 16Schoolbook	38262.3	11599.7	2376
1Toom-4 + 3Karatsuba + 8Schoolbook	33512.3	10209.3	2376
1Toom-4 + 2Karatsuba + 16Schoolbook	29241.1	8858.1	2376
2Toom-4 + 1Karatsuba + 8Schoolbook	26020.1	7927.9	2376
2Toom-4 + 16Schoolbook	22241.1	6741.7	2376

Table 9: **Average Performance for Different Non-striding Multiplication Strategies** ($n = 512$)

Multiplication Strategy	Avg CPU Cycles	Avg Time (ns)	Avg Memory (KB)
6Karatsuba + 8Schoolbook	154587.8	45838.3	2504
1Toom-4 + 2Karatsuba + 32Schoolbook	115760.4	35203.6	2504
5Karatsuba + 16Schoolbook	112607.0	34140.2	2504
1Toom-4 + 4Karatsuba + 8Schoolbook	96014.2	28610.4	2504
1Toom-4 + 3Karatsuba + 16Schoolbook	85933.3	26072.4	2504
2Toom-4 + 32Schoolbook	76753.0	23169.8	2504
3Toom-4 + 8Schoolbook	67448.6	20462.2	2504
2Toom-4 + 1Karatsuba + 16Schoolbook	67100.0	20378.4	2504

Table 10: **Average Performance for Striding Multiplication Strategies for all Security Versions**

Parameter Set	Striding Multiplication Strategy	Avg CPU Cycles	Avg Time (ns)	Avg Memory (KB)
$n = 128$	2Toom-4 + 8Schoolbook	7952.7	2414.4	2364
$n = 256$	2Toom-4 + 16Schoolbook	26231.3	7822.4	2528
	2Toom-4 + 1Karatsuba + 8Schoolbook	23129.3	7129.8	2372
$n = 512$	2Toom-4 + 1Karatsuba + 16Schoolbook	75986.1	23324.2	2552
	3Toom-4 + 8Schoolbook	57947.6	18180.9	2372

5.1.2 Reference and Optimized Implementation of ATLAS

We provide both a portable reference implementation and an AVX2 optimized implementation targeting **x86-64** platforms. The performance of the reference implementation is reported in Table 11, which presents the median and average clock cycles for `sig_keygen`, `sig_sign`, and `sig_verify`.

The reference implementation avoids secret dependent branching and secret dependent memory accesses in all computations involving long term secret material. Since the moduli used in **ATLAS** are powers of two, modular reduction is implemented entirely through bitmasking operations of the form $\mathbf{x} \& (\mathbf{q}-1)$, eliminating the need for division operations as well as Montgomery and Barrett reduction routines. In addition, all supporting arithmetic routines are implemented using branch free arithmetic. Whenever the information revealed is independent of the secret key, constant time execution is not enforced. In particular, the rejection conditions during signing are permitted to reveal which condition triggered the rejection, since the rejection probability depends only on the public parameters γ_1 and β_1 . Likewise, revealing the challenge $c \leftarrow \mathbf{H}(\mu, |, \mathbf{w}_1)$ during rejected signing attempts does not compromise security because $\mathbf{w}_1$ is computed from the public masking vector $\mathbf{y}$ and possesses high min entropy when $\mathbf{H}$ is modeled as a random oracle.

The optimized implementation improves performance through two complementary techniques. First, the degree 8 schoolbook multiplication together with the principal arithmetic routines used during signing and verification, including `decompose_keygen`, `decompose_sign`, `poly_freeze`, `poly_add`, `poly_sub`, `poly_shiftl`, `poly_neg`, `poly_freeze`, `make_hint`, and `use_hint`, are implemented using AVX2 intrinsics, enabling multiple 32 bit coefficients to be processed simultaneously within a single instruction. Second, polynomial multiplication is accelerated using a strided hierarchical decomposition based on Toom-Cook and Karatsuba multiplication. The striding technique performs negacyclic reduction modulo $x^n + 1$ implicitly throughout the recursive decomposition, thereby eliminating a separate reduction stage. After evaluating several decomposition strategies, the following configurations were found to provide the best practical performance: two levels of Toom 4 followed by degree 8 schoolbook multiplication for $n = 128$; two levels of Toom 4, one level of Karatsuba, and degree 8 schoolbook multiplication for $n = 256$; and three levels of Toom 4 followed by degree 8 schoolbook multiplication for $n = 512$. The performance of the optimized implementation is reported in Table 11, which lists the median and average clock cycles for `sig_keygen`, `sig_sign`, and `sig_verify`.

Table 11: CPU cycle counts for the reference (Ref) and optimized (Opt) implementations of ATLAS.

Security Level	Impl.	sig_keygen		sig_sign		sig_verify	
		Med	Avg	Med	Avg	Med	Avg
ATLAS-128	Ref	1,817,945	1,813,375	4,614,995	5,386,122	1,913,472	1,909,422
	Opt	1,758,848	1,766,088	3,356,227	3,770,551	1,797,703	1,805,581
ATLAS-192	Ref	4,353,556	4,346,517	10,281,772	12,286,833	4,527,209	4,536,784
	Opt	4,129,737	4,145,821	7,151,899	8,180,431	4,256,300	4,272,031
ATLAS-256	Ref	3,397,086	3,441,651	8,749,181	10,388,201	3,634,396	3,681,474
	Opt	2,906,423	2,921,922	6,598,731	7,723,028	3,054,224	3,071,332
ATLAS-512	Ref	9,282,292	9,360,891	18,699,235	23,689,208	9,902,510	9,991,888
	Opt	7,137,998	7,170,600	13,032,107	16,424,152	7,387,980	7,421,373

5.2 Performance Analysis, Considerations and Trade-offs

The performance characteristics of ATLAS are directly derived from the design and implementation choices described above. ATLAS achieves relatively low computational complexity compared to other LWE-based schemes through three main design decisions. First, the use of power-of-two moduli $q = 2^{23}$ and $p = 2^{18}$ eliminates rejection sampling during matrix generation and reduces modular reduction to a single bitwise AND with $(q - 1)$, yielding a constant-time, deterministic implementation at minimal instruction cost. Second, by avoiding the NTT entirely in favour of strided Toom-Cook and Karatsuba decomposition, ATLAS sidesteps the overhead of bit-reversal permutations and butterfly networks; the multiplication strategy remains adaptable to the target platform, as demonstrated by the AVX2 implementation, where SIMD vectorisation provides consistent throughput gains across all security levels. Third, by varying both the module dimensions (k, l) and the degree of polynomial n between security levels, ATLAS maintains a compact memory footprint at each level; in particular, $n = 128$ at Level 1 allows intermediate vectors to reside in high-speed registers, directly reducing signing and verification latency. The concrete impact of these decisions is reflected in Table 11. The AVX2-optimized implementation delivers consistent improvements across all parameter sets, with the most substantial gains observed in **sig_sign**. At the 128-bit and 192-bit security levels, polynomial multiplication is performed using an AVX2-vectorised schoolbook implementation, which processes multiple 32-bit coefficients simultaneously within a single instruction. This approach proves particularly effective at these parameter sets because the striding technique yields greater efficiency as the polynomial size increases: larger polynomials provide sufficient arithmetic intensity for the strided schoolbook kernel to amortise its SIMD setup overhead across a greater number of coefficients, resulting in higher effective throughput. Consequently, **sig_sign** improves by approximately $1.38\times$ and $1.44\times$ at the 128-bit and 192-bit levels respectively (around 27% and 30% cycle reduction), while **sig_keygen** and **sig_verify** also improve modestly by up to $1.05\times$ and $1.06\times$ respectively. At the 256-bit and 512-bit security levels, polynomial multiplication is instead handled using strided Toom-Cook combined with Karatsuba decomposition, which reduces the asymptotic multiplication complexity and scales more favourably with the larger ring degrees at these parameter sets. By recursively decomposing the polynomial product into smaller subproblems before applying the base schoolbook kernel, this strategy significantly reduces the total number of coefficient-level operations, yielding more uniformly distributed gains across all three operations: **sig_sign** improves by $1.33\times$ and

$1.43\times$ (around 25% and 30%), `sig_verify` by $1.19\times$ and $1.34\times$ (around 16% and 25%), and `sig_keygen` by $1.17\times$ and $1.30\times$ (around 14% and 23%). Overall, these results confirm that the combination of AVX2-vectorised schoolbook arithmetic at the lower security levels and strided Toom-Cook–Karatsuba decomposition at the higher security levels together deliver meaningful and consistent performance gains across all supported parameter sets, with the signing operation benefiting most significantly at every security level.

5.3 Hardware Implementation

The design of **ATLAS** incorporates several algorithmic choices that make it well suited for hardware implementations on resource-constrained IoT devices. First, unlike MLWE-based Dilithium, the underlying MLWR formulation eliminates the need for explicitly generating the error vector. Instead, the required noise is introduced through a deterministic rounding operation, which is significantly simpler to implement in hardware and incurs negligible computational overhead. Furthermore, the use of a power-of-two modulus enables modular reduction to be implemented using simple bit-shift operations, thereby eliminating costly integer division circuitry and making the overall design more amenable to efficient hardware realization.

The proposed striding polynomial multiplication strategy is also motivated by prior hardware-oriented studies. The work in [6] demonstrates that striding Toom-4 and Karatsuba polynomial multiplication achieve lower execution time and reduced memory usage than conventional hybrid Toom-4/Karatsuba decomposition on embedded Cortex-M4 processors. Similarly, [14] reports a silicon-validated ASIC implementation of the Saber key encapsulation mechanism, where optimized memory operations enabled by striding Toom-Cook multiplication reduce memory footprint, area, and overall energy consumption. These observations indicate that the proposed multiplication framework is well suited for efficient hardware implementations of **ATLAS**.

We are currently exploring the implementation of Compute-in-Memory (CiM) architectures to accelerate the base-level degree-8 schoolbook polynomial multiplication. Since all higher-level Toom-4 and Karatsuba decompositions ultimately invoke the base schoolbook multiplier, accelerating this operation using CiM-based matrix-vector multiplication techniques can significantly improve the overall performance and energy efficiency of polynomial multiplication in hardware.

6 CONCLUSION

In the section, we explicitly outline the advantages of **ATLAS** regarding its optimized storage and high-throughput performance, as well as the practical limitations and design constraints identified during its development.

6.1 Advantages of ATLAS

- **Algorithmic Determinism:** By grounding security in the MLWR problem, **ATLAS** removes the volatility associated with error sampling. Replacing discrete Gaussian or binomial distributions with a deterministic rounding function significantly hardens the implementation against timing attacks and reduces the entropy requirements for the pseudorandom number generator.

- **Implicit Modular Arithmetic:** The use of power-of-two moduli transforms modular reduction from a costly division operation into a simple bitwise truncation. This architectural choice ensures that the scheme remains highly efficient on both high-end processors and 8-bit or 16-bit microcontrollers.
- **Implementation Versatility:** Unlike schemes locked into a specific NTT, ATLAS provides a multiplication-friendly framework. Designers can deploy Toom-Cook 3-way for high-end desktop CPUs or Karatsuba for low-gate-count FPGAs, allowing for platform-specific optimization that is not tethered to prime-modulus constraints.
- **Masking Efficiency:** Power-of-two arithmetic is a natural fit for side-channel resistance. Masking ATLAS is substantially simpler than masking schemes like Dilithium [13] or Kyber [8], as it avoids the complex Boolean-to-Arithmetic conversions required for rejection sampling and prime-field reductions.
- **Compact Footprint:** The rounding mechanism naturally acts as a compression tool. By discarding the least significant bits during public key and signature generation, ATLAS achieves a tighter byte-representation than traditional LWE schemes, making it ideal for bandwidth-constrained network protocols.

6.2 Constraints and Tradeoffs

- **Asymptotic Complexity Shift:** The lack of a prime modulus means the standard, highly-optimized NTT cannot be used. While Karatsuba and Toom-Cook are highly performant in practice, they possess a higher asymptotic complexity ($O(n^{1.58})$ or $O(n^{1.46})$ respectively) compared to the $O(n \log n)$ of the NTT. In very high-security tiers where n is large, this may result in a computational delta.
- **Memory vs. Speed Tradeoff:** Without the NTT's in-place computation benefits, some alternative multiplication algorithms may require additional temporary buffer space. Implementers must balance the memory footprint of these algorithms when deploying on resource-constrained IoT sensors.
- **Reduction Tightness:** While MLWR is a robust hard problem, the reduction from worst-case lattice problems to LWR can be technically more complex than the reduction to LWE due to the deterministic nature of the rounding error. ATLAS compensates for this with conservative parameter selection to ensure 128-bit to 512-bit classical security.
- **Large Signature Size:** While ATLAS optimizes key storage, its signature size remains comparatively large. This is an inherent characteristic and a known challenge in most lattice-based signature schemes.

6.3 Additional Security Level: 384-bit.

In addition to the implemented parameter sets, ATLAS also supports a parameter set targeting the optional 384-bit classical security level specified in the NGCC requirements. The parameters were derived using the same methodology adopted for the implemented parameter sets, and their estimated concrete

security against the standard lattice attacks is summarized in Table 12. Since this security level is optional, we do not include a dedicated software implementation or performance benchmarks in the current specification. A complete implementation together with a more comprehensive experimental evaluation remains future work. The parameters are $n = 256$, $k = 13$, $l = 10$, $\beta_1 = 70$, $\beta_2 = 18$, $q = 2^{25}$, $p = 2^{21}$, $\eta = 8$, $\gamma_1 = 2^{21}$, $\bar{\gamma}_1 = 2^{17}$, $\gamma_2 = 1048576$, $\bar{\gamma}_2 = 2^{16}$, $\zeta = 3186704.0$, $\zeta' = 4194306$, $d = 10$, $\kappa = 133$, $\omega = 168$.

Table 12: Concrete security estimates for the optional 384-bit ATLAS parameter set.

Attack	Block Size	Classical Cost	Quantum Cost
SIS	1496	437	397
MSIS	1443	422	382
Primal	1441	421	382
Dual	1435	419	380

7 BIBLIOGRAPHY

References

- [1] TIMECOP: Automatic detection of timing side channels. <https://www.post-apocalyptic-crypto.org/timecop/>, 2020. Accessed: 2026-06-27.
- [2] Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, and Daniel Smith-Tone. Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. Online. Accessed 26th January, 2024, 2022.
- [3] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber (version 3.02) – Submission to round 3 of the NIST post-quantum cryptography standardization process. Available online at <https://pq-crystals.org>, August 2021.
- [4] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, pages 719–737, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [5] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA ’16, page 10–24, USA, 2016. Society for Industrial and Applied Mathematics.
- [6] Hanno Becker, Jose Maria Bermudo Mera, Angshuman Karmakar, Joseph Yiu, and Ingrid Verbauwhede. Polynomial multiplication on embedded vector architectures. Cryptology ePrint Archive, Paper 2021/998, 2021.

- [7] Marco Bodrato and Alberto Zanzi. Integer and polynomial multiplication: towards optimal toom-cook matrices. In *Proceedings of the 2007 International Symposium on Symbolic and Algebraic Computation*, ISSAC '07, page 17–24, New York, NY, USA, 2007. Association for Computing Machinery.
- [8] Joppe Bos, Leo Ducas, Eike Kiltz, T Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. Crystals - kyber: A cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367, 2018.
- [9] Olivier Bronchain and Gaëtan Cassiers. Bitslicing arithmetic/boolean masking conversions for fun and profit with application to lattice-based kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):553–588, 2022.
- [10] Jung Hee Cheon, Hyeongmin Choe, Julien Devevey, Tim Güneysu, Dongyeon Hong, Markus Krausz, Georg Land, Junbum Shin, Damien Stehlé, and MinJune Yi. Haetae: Hyperball bimodal module rejection signature scheme. *Supporting Document of Korean Post-Quantum Cryptography Competition*, 2023.
- [11] Cas Cremers, Samed Düzl , Rune Fiedler, Marc Fischlin, and Christian Janson. BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures. In *42nd IEEE Symposium on Security and Privacy (S&P)*, pages 1696–1714. IEEE, 2021.
- [12] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *Progress in Cryptology – AFRICACRYPT 2018*, pages 282–305, Cham, 2018. Springer International Publishing.
- [13] L o Ducas, Eike Kiltz, Tancre de Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehl . Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, Feb. 2018.
- [14] Archisman Ghosh, Jose Maria Bermudo Mera, Angshuman Karmakar, Debayan Das, Santosh Ghosh, Ingrid Verbauwhede, and Shreyas Sen. A 334 μ w 0.158 mm² asic for post-quantum key-encapsulation mechanism saber with low-latency striding toom-cook multiplication. *IEEE Journal of Solid-State Circuits*, 58(8):2383–2398, 2023.
- [15] Institute of Commercial Cryptography Standards (ICCS). Call for proposals for the next-generation public-key cryptographic algorithms, October 2025.
- [16] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. In *Advances in Cryptology – EUROCRYPT 2018*, volume 10822 of *Lecture Notes in Computer Science*, pages 552–586. Springer, 2018.
- [17] KpqC Research Center. Selected algorithms from the KpqC competition round 2. Post-Quantum Cryptography Research Center, South Korea, January 2025. Official Announcement of National PQC Standards.
- [18] Suparna Kundu, Jan-Pieter D’Anvers, Michiel Van Beirendonck, Angshuman Karmakar, and Ingrid Verbauwhede. Higher-order masked saber. In Clemente Galdi and Stanislaw Jarecki, editors, *Security and Cryptography for Networks - 13th International Conference, SCN 2022, Amalfi, Italy*,

September 12-14, 2022, *Proceedings*, volume 13409 of *Lecture Notes in Computer Science*, pages 93–116. Springer, 2022.

- [19] Suparna Kundu, Archisman Ghosh, Angshuman Karmakar, Shreyas Sen, and Ingrid Verbauwhede. Rudraksh: A compact and lightweight post-quantum key-encapsulation mechanism. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(2):647–680, 2025.
- [20] Suparna Kundu, Quinten Norga, Angshuman Karmakar, Shreya Gangopadhyay, Jose Maria Bermudo Mera, and Ingrid Verbauwhede. Scabbard: An exploratory study on hardware aware design choices of learning with rounding-based key encapsulation mechanisms. *ACM Trans. Embed. Comput. Syst.*, 24(1):10:1–10:40, 2025.
- [21] Thijs Laarhoven. *Search problems in cryptography: from fingerprinting to lattice sieving*. Phd thesis 1 (research tu/e / graduation tu/e), Mathematics and Computer Science, February 2016. Proefschrift.
- [22] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.
- [23] Hiroki Okada, Atsushi Takayasu, Kazuhide Fukushima, Shinsaku Kiyomoto, and Tsuyoshi Takagi. A compact digital signature scheme based on the module-lwr problem. In Weizhi Meng, Dieter Gollmann, Christian D. Jensen, and Jianying Zhou, editors, *Information and Communications Security*, pages 73–90, Cham, 2020. Springer International Publishing.
- [24] Prasanna Ravi, Debapriya Basu Roy, Shivam Bhasin, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. Number "not used" once - practical fault attack on pqm4 implementations of NIST candidates. In Ilia Polian and Marc Stöttinger, editors, *Constructive Side-Channel Analysis and Secure Design - 10th International Workshop, COSADE 2019, Darmstadt, Germany, April 3-5, 2019, Proceedings*, volume 11421 of *Lecture Notes in Computer Science*, pages 232–250. Springer, 2019.
- [25] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.
- [26] C. P. Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1):181–199, August 1994.
- [27] State Commercial Cryptography Administration. Specification of sm3 cryptographic hash algorithm. Technical report, Office of Security Commercial Code Administration, 2010. Official Standard Release.

A MISSING PROOFS

A.1 Min-Entropy

The security reduction for Fiat–Shamir with Aborts signatures [13] relies on the underlying identification protocol having sufficiently high min-entropy. Intuitively, a high min-entropy guarantees that the prover’s commitment is distributed over a large support, making any particular commitment unlikely to occur. This property is essential in the random-oracle security reduction and directly affects its tightness. Since

ATLAS is obtained by applying the Fiat–Shamir with Aborts transformation to an identification protocol, we next establish a lower bound on its min-entropy.

Definition A.1 (Min-Entropy). *Let W be a random variable sampled from a discrete distribution $\mathcal{D}$. If the most likely outcome of W occurs with probability $2^{-\tau}$, then the min-entropy of W is defined as $H_\infty(W) = -\log_2(\max_w \Pr[W = w]) = \tau$. Furthermore, let ID be a canonical identification scheme with key-generation algorithm IDS.Gen and prover commitment algorithm IDS.P. We say that ID has τ bits of min-entropy if*

$$\Pr_{(\text{pk}, \text{sk}) \leftarrow \text{IDS.Gen}(\text{par})} [H_\infty(W) \geq \tau \mid (W, \text{St}) \leftarrow \text{P}_1(\text{sk})] \geq 1 - 2^{-\tau}.$$

To derive the min-entropy bound, we first establish an upper bound on the probability of observing any fixed commitment component $\mathbf{w}_1$. This bound forms the key ingredient in proving the min-entropy of the underlying identification protocol.

Lemma A.1. *Let $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}$ be chosen uniformly at random. For a fixed value $\mathbf{w}_1$, define*

$$pr_{(\mathbf{w}_1, \mathbf{A})} := \Pr_{\mathbf{y} \leftarrow S_{\gamma_1-1}^l} \left[\text{decompose_sign} \left(\left\lfloor \frac{p}{q} \mathbf{A} \mathbf{y} \right\rfloor, 2\bar{\gamma}_2 \right) = (\mathbf{w}_1, \cdot) \right].$$

Then,

$$\Pr_{\mathbf{A} \leftarrow \mathcal{R}_q^{k \times l}} \left[\forall \mathbf{w}_1 : pr_{(\mathbf{w}_1, \mathbf{A})} \leq \left(\frac{2\bar{\gamma}_2 + 1}{2\gamma_1 - 1} \right)^n \right] > 1 - \left(\frac{n}{q} \right)^{kl}.$$

Proof. For a uniformly random polynomial $f \in \mathcal{R}_q$, the probability of being invertible is $(1 - \frac{1}{q})^n > 1 - \frac{n}{q}$. Consequently, for a random matrix $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}$, the probability that none of its entries is invertible is at most $(n/q)^{kl}$. Equivalently, with probability greater than $1 - (n/q)^{kl}$. There exists at least one invertible entry in $\mathbf{A}$.

Thus, it remains to show that every matrix $\mathbf{A}$ containing an invertible polynomial satisfies $pr_{(\mathbf{w}_1, \mathbf{A})} \leq \left(\frac{2\bar{\gamma}_2 + 1}{2\gamma_1 - 1} \right)^n$ for all $\mathbf{w}_1$. Let $[f_0, f_1, \dots, f_{l-1}]$ denote a row of $\mathbf{A}$ containing an invertible entry. Without loss of generality, we assume that f_0 is invertible. The remainder of the proof focuses on this row.

To establish the desired bound on $pr_{(\mathbf{w}_1, \mathbf{A})}$, it suffices to show that, for every fixed $\mathbf{w}_1$,

$$\Pr_{\mathbf{y} \leftarrow S_{\gamma_1-1}^l} \left[\text{decompose_sign} \left(\left\lfloor \frac{p}{q} \sum_{i=0}^{l-1} f_i y_i \right\rfloor, 2\bar{\gamma}_2 \right) = (\mathbf{w}_1, \cdot) \right] \leq \left(\frac{2\bar{\gamma}_2 + 1}{2\gamma_1 - 1} \right)^n.$$

Define $\mathcal{W}_{\mathbf{w}_1} := \{\mathbf{w} \in \mathcal{R}_p : \text{decompose_sign}(\mathbf{w}, 2\bar{\gamma}_2) = (\mathbf{w}_1, \cdot)\}$. That is, $\mathcal{W}_{\mathbf{w}_1}$ consists of all values whose high-order component is equal to the fixed value $\mathbf{w}_1$. Since each coefficient admits at most $2\bar{\gamma}_2 + 1$ possible low-order values, we have $|\mathcal{W}_{\mathbf{w}_1}| \leq (2\bar{\gamma}_2 + 1)^n$. Using the definition of $\mathcal{W}_{\mathbf{w}_1}$, the probability under consideration can be expressed as

$$\Pr_{\mathbf{y} \leftarrow S_{\gamma_1-1}^l} \left[\left\lfloor \frac{p}{q} \sum_{i=0}^{l-1} f_i y_i \right\rfloor \in \mathcal{W}_{\mathbf{w}_1} \right].$$

Since f_0 is invertible, we may isolate y_0 and rewrite the above probability as

$$\Pr_{\mathbf{y} \leftarrow \mathcal{S}_{\gamma_1-1}^l} \left[y_0 \in f_0^{-1} \left(\frac{q}{p} (\mathcal{W}_{\mathbf{w}_1} - \boldsymbol{\xi}) - \sum_{i=1}^{l-1} f_i y_i \right) \right],$$

where $\boldsymbol{\xi}$ denotes the rounding error introduced by the operation $\lfloor \frac{p}{q}(\cdot) \rfloor$.

Since f_0 is invertible, multiplication by f_0^{-1} defines a bijection on $\mathcal{R}_q$. Furthermore, translations by $\boldsymbol{\xi}$ and $\sum_{i=1}^{l-1} f_i y_i$ preserve cardinality. Therefore,

$$\left| f_0^{-1} \left(\frac{q}{p} (\mathcal{W}_{\mathbf{w}_1} - \boldsymbol{\xi}) - \sum_{i=1}^{l-1} f_i y_i \right) \right| \leq (2 \frac{q}{p} \bar{\gamma}_2 + 1)^n = (2\gamma_2 + 1)^n.$$

On the other hand, $(\mathbf{y})_i$ is sampled uniformly from $\mathcal{S}_{\gamma_1-1}^l$, which contains exactly $(2\gamma_1 - 1)^n$ elements. It follows that

$$\Pr_{\mathbf{y} \leftarrow \mathcal{S}_{\gamma_1-1}^l} \left[\left\lfloor \frac{p}{q} \sum_{i=0}^{l-1} f_i y_i \right\rfloor \in \mathcal{W}_{\mathbf{w}_1} \right] \leq \left(\frac{2\gamma_2 + 1}{2\gamma_1 - 1} \right)^n.$$

□

A.2 Proof of EUF-KOA Security

In this appendix, we provide the complete proof of Lemma 3.1, establishing the EUF-KOA security of the proposed ATLAS signature scheme. The proof of Theorem 3.1 can be established using techniques analogous to those employed in Dilithium [13]. For convenience, we restate the lemma.

Lemma 3.1 (EUF-KOA Security). *Suppose $\mathcal{QA}$ is a quantum adversary against the EUF-KOA-security of the ATLAS signature scheme that issues q_H queries to the quantum random oracle $|H\rangle$. Then, there exist quantum adversaries $\mathcal{B}$ and $\mathcal{C}$ that solve the MLWR problem with parameters $\mathbf{pp}_1 = \{k, l, S\}$ and the SelfTargetMSIS problem with parameters $\mathbf{pp}_2 = \{H, k, l + 1, \zeta\}$, respectively, such that the following holds.*

$$\text{Adv}_{\text{ATLAS}}^{\text{EUF-KOA}}(\mathcal{QA}) \leq \text{Adv}_{\text{MLWR}}^{\mathbf{pp}_1}(\mathcal{B}) + \text{Adv}_{\text{SelfTargetMSIS}}^{\mathbf{pp}_2}(\mathcal{C})$$

Furthermore, the time complexity is bounded by $\text{Time}(\mathcal{B}) = \text{Time}(\mathcal{C}) = \text{Time}(\mathcal{QA}) + q_H$.

Proof. We prove the lemma through a sequence of hybrid games, **Game₀** and **Game₁**, each differing from the previous in exactly one step. We denote $\Pr[\text{Game}_i]$ for the probability that $\mathcal{QA}$ wins **Game_i**.

Game₀ (The original EUF-KOA game). The challenger generates a ATLAS public key $pk = (\mathbf{A}, \mathbf{t})$ by sampling $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}$, $\mathbf{s}_1 \xleftarrow{\$} S$, and computing $\mathbf{t} = \lfloor \frac{p}{q} \mathbf{A} \mathbf{s}_1 \rfloor \in \mathcal{R}_p^k$. Then pk is given to $\mathcal{QA}$, who may query the quantum random oracle $|H\rangle$ up to q_H times. $\mathcal{QA}$ wins if it outputs a valid forgery (M, σ) with $\sigma = (\mathbf{z}, \mathbf{h}, c)$ that passes the verification check and whose message M was never signed. By definition,

$$\Pr[\text{Game}_0] = \text{Adv}_{\text{ATLAS}}^{\text{EUF-KOA}}(\mathcal{QA}).$$

Game₁ (Switch to a pseudorandom public key). In this game, we replace the honestly generated $\mathbf{t}$ with the high-order part of a uniformly random $\mathbf{t}' \xleftarrow{\$} \mathcal{R}_p^k$. Concretely, the challenger:

1. Receives an **SelfTargetMSIS** instance $[\mathbf{A}, \mathbf{t}'] \in \mathcal{R}_q^{k \times l} \times \mathcal{R}_q^k$ from the **SelfTargetMSIS** challenger.
2. Decomposes $\mathbf{t}'$ into its high- and low-order parts: $\mathbf{t}' = \frac{q}{p}\mathbf{t} + \mathbf{v}$, where $\mathbf{t} \in \mathcal{R}_p^k$ and $\mathbf{v} \in \mathcal{R}_{q/p}^k$. Because $\mathbf{t}'$ is uniform over $\mathcal{R}_q^k$, the distribution of $\mathbf{t}$ is also uniform over $\mathcal{R}_p^k$.
3. Sends $(\mathbf{A}, \mathbf{t})$ to $\mathcal{QA}$ as the public key and simulates the quantum random oracle $|\mathbf{H}\rangle$.

Transition analysis. The only difference between **Game**₀ and **Game**₁ is that the public key is now derived from a uniform $\mathbf{t}' \in \mathcal{R}_q^k$ rather than being computed via the **MLWR** rounding step. By the hardness of **MLWR**_{pp₁}, these two distributions are computationally indistinguishable. Any adversary $\mathcal{QA}$ that distinguishes them can be used to build an **MLWR** solver $\mathcal{B}$, so

$$\left| \Pr[\text{Game}_1] - \Pr[\text{Game}_0] \right| \leq \text{Adv}_{\text{MLWR}}^{\text{pp}_1}(\mathcal{B}).$$

Game₂ (**Reducing a forgery to an SelfTargetMSIS solution**). This game is identical to **Game**₁. However, we now show that any valid forgery $\mathcal{QA}$ produces can be converted into a solution of the underlying **SelfTargetMSIS**_{pp₂} instance, so the winning probability transfers to $\mathcal{C}$.

Suppose $\mathcal{QA}$ outputs a valid forgery (M, σ) with $\sigma = (\mathbf{z}, \mathbf{h}, c)$. By the correctness of the verification algorithm, the forgery satisfies $c = \mathbf{H}(\mu \parallel \mathbf{w}_1)$, where $\mathbf{w}_1 = \text{use_hint}(\mathbf{h}, \lfloor \frac{p}{q} \mathbf{A} \mathbf{z} \rfloor - c \mathbf{t}_1 \cdot 2^{pk_{\text{drop}}}, 2\bar{\gamma}_2)$, and the norm constraints $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$ and $\text{HW}(\mathbf{h}) < \omega$ are satisfied.

Step 1: Unpacking use_hint. By definition of **use_hint**, for a vector $\mathbf{u}$ with $\|\mathbf{u}\|_\infty \leq 2\bar{\gamma}_2 + 1$ we can write

$$2\bar{\gamma}_2 \cdot \text{use_hint}(\mathbf{h}, \lfloor \frac{p}{q} \mathbf{A} \mathbf{z} \rfloor - c \mathbf{t}_1 \cdot 2^{pk_{\text{drop}}}, 2\bar{\gamma}_2) = \lfloor \frac{p}{q} \mathbf{A} \mathbf{z} \rfloor - c \mathbf{t}_1 \cdot 2^{pk_{\text{drop}}} + \mathbf{u}. \quad (15)$$

Step 2: Using the key-generation relation. From the key-generation algorithm, we have $\lfloor \frac{p}{q} \mathbf{A} \mathbf{s}_1 \rfloor = \mathbf{t} = \mathbf{t}_1 \cdot 2^{pk_{\text{drop}}} + \mathbf{t}_0$, where $\|\mathbf{t}_0\|_\infty \leq 2^{pk_{\text{drop}}-1}$. Substituting this value into (15) and letting $\boldsymbol{\xi} := \lfloor \frac{p}{q} \mathbf{A} \mathbf{z} \rfloor - \frac{p}{q} \mathbf{A} \mathbf{z}$ denote the rounding error (so $\|\boldsymbol{\xi}\|_\infty \leq 1$), one obtains

$$\lfloor \frac{p}{q} \mathbf{A} \mathbf{z} \rfloor - c \mathbf{t}_1 \cdot 2^{pk_{\text{drop}}} + \mathbf{u} = \frac{p}{q} \left[\mathbf{A}, \frac{q}{p} \mathbf{t} + \mathbf{v}, \mathbf{I}_k \right] \begin{bmatrix} \mathbf{z} \\ -c \\ \frac{q}{p} \mathbf{u}' \end{bmatrix}, \quad (16)$$

where we set $\mathbf{u}' := c(\mathbf{t}_0 + \frac{p}{q} \mathbf{v}) + \mathbf{u} + \boldsymbol{\xi}$.

Step 3: Bounding $\|\mathbf{u}'\|_\infty$. Since $\mathbf{v}$ is uniform over $\mathcal{R}_{q/p}^k$, we have $\|\mathbf{t}_0 + \frac{p}{q} \mathbf{v}\|_\infty \leq 2^{pk_{\text{drop}}-1} + 1$. Using $\|c\|_1 = \kappa$, $\|\mathbf{u}\|_\infty \leq 2\bar{\gamma}_2 + 1$, and $\|\boldsymbol{\xi}\|_\infty \leq 1$, we can bound the norm of $\mathbf{u}'$ as:

$$\|\mathbf{u}'\|_\infty \leq \kappa(2^{pk_{\text{drop}}-1} + 1) + 2\bar{\gamma}_2 + 1 < 4\bar{\gamma}_2 = 4\frac{p}{q}\gamma_2.$$

Step 4: Assembling the SelfTargetMSIS solution. Setting $\mathbf{t}' := \frac{q}{p}\mathbf{t} + \mathbf{v}$ and $\mathbf{H}'(\mu \parallel x) := \mathbf{H}(\mu \parallel \frac{1}{2\bar{\gamma}_2}x)$, the forgery satisfies

$$\mathbf{H}' \left(\mu \parallel \left[\mathbf{A}, \mathbf{t}', \mathbf{I}_k \right] \begin{bmatrix} \mathbf{z} \\ -c \\ \frac{q}{p} \mathbf{u}' \end{bmatrix} \right) = c, \quad (17)$$

with $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$, $\|c\|_\infty = 1$, and $\|\mathbf{u}'\|_\infty < 4\frac{p}{q}\gamma_2$. Since $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}$ and $\mathbf{t}' \xleftarrow{\$} \mathcal{R}_q^k$ are both uniformly distributed, $\mathbf{y} := [\mathbf{z}^\top \mid c \mid \frac{q}{p}\mathbf{u}'^\top]^\top$ constitutes a valid solution to the $\text{SelfTargetMSIS}_{\text{pp}_2}$ instance held by $\mathcal{C}$. Therefore

$$\Pr[\text{Game}_2] \leq \text{Adv}_{\text{SelfTargetMSIS}}^{\text{pp}_2}(\mathcal{C}).$$

Combining the games. Chaining the three games together gives

$$\begin{aligned} \text{Adv}_{\text{ATLAS}}^{\text{EUF-KOA}}(\mathcal{QA}) &= \Pr[\text{Game}_0] \\ &\leq \Pr[\text{Game}_1] + \text{Adv}_{\text{MLWR}}^{\text{pp}_1}(\mathcal{B}) && \text{(by the MLWR transition)} \\ &\leq \text{Adv}_{\text{SelfTargetMSIS}}^{\text{pp}_2}(\mathcal{C}) + \text{Adv}_{\text{MLWR}}^{\text{pp}_1}(\mathcal{B}), && \text{(by the SelfTargetMSIS reduction)} \end{aligned}$$

which is exactly the bound stated in the lemma. $\square$

B MISSING ALGORITHMS

B.1 Polynomial Multiplication

This section presents the classical polynomial multiplication algorithms used in the implementation of ATLAS. Algorithm 15 describes the standard quadratic-complexity schoolbook multiplication for two degree-8 polynomials. This routine serves as the base multiplication for all the finalized hybrid polynomial multiplication strategies employing striding Toom-4 and Karatsuba decompositions.

Algorithm 15 8×8 Schoolbook Polynomial Multiplication over $\mathcal{R}_q[x]/(x^8 + 1)$

Input: Two degree-8 polynomials $A(x)$ and $B(x)$
Output: $C(x) = A(x) * B(x)$ over $\mathcal{R}_q[x]/(x^8 + 1)$

```

1: Initialize  $C'(x) \leftarrow 0$ 
2: for  $i = 0$  to  $7$  do
3:   for  $j = 0$  to  $7$  do
4:      $C'_{i+j} \leftarrow C'_{i+j} + A_i B_j$ 
5:   end for
6: end for
7: for  $i = 0$  to  $6$  do
8:    $C_i \leftarrow C'_i - C'_{i+8}$ 
9: end for
10:  $C_7 \leftarrow C'_7$ 
11: return  $C(x)$ 

```

Algorithm 16 presents the conventional or non-striding Karatsuba polynomial multiplication algorithm, while Algorithms 17 and 18 provide its evaluation and interpolation subroutines, respectively. Although the final implementation employs striding Karatsuba, these classical subroutines form the basis of the striding evaluation and interpolation procedures and are invoked as part of the hybrid multiplication strategy for the $n = 256$ parameter set of ATLAS.

Algorithm 16 Karatsuba Polynomial Multiplication**Input:** Two polynomials $A(x)$ and $B(x)$ of degree n , Base threshold n_{base} **Output:** $C(x) = A(x) * B(x)$

```

1: if  $n \leq n_{\text{base}}$  then
2:   return Schoolbook $_{n_{\text{base}} \times n_{\text{base}}}(A, B)$  ▷ Base Case
3: end if
4:  $A(x) = A_1 \cdot y + A_0$ , where  $y = x^{n/2}$  ▷ Decomposition of  $A$ 
5:  $B(x) = B_1 \cdot y + B_0$  ▷ Decomposition of  $B$ 
6:  $w_0 = A(0) * B(0) = A_0 * B_0$  ▷ Evaluation at  $y = 0$ 
7:  $w_2 = A(\infty) * B(\infty) = A_1 * B_1$  ▷ Evaluation at  $y = \infty$ 
8:  $w_1 = A(1) * B(1) = (A_0 + A_1) * (B_0 + B_1)$  ▷ Evaluation at  $y = 1$ 
9:  $w_1 = w_1 - w_0 - w_2$ 
10: return  $C(x) = w_2 \cdot y^2 + w_1 \cdot y + w_0$  ▷ Interpolation

```

Algorithm 17 Classical Karatsuba Evaluation**Input:** Polynomial $A(x)$ of degree n **Output:** Evaluations $[A^{(0)}, A^{(1)}, A^{(2)}]$

```

1:  $A(y) = A_1 y + A_0$ ,  $y = x^{n/2}$  ▷ Split  $A(x)$  into two polynomials of size  $n/2$ 
2:  $A^{(0)} \leftarrow A(0) = A_0$  ▷ Evaluation at  $y = 0$ 
3:  $A^{(1)} \leftarrow A(1) = A_0 + A_1$  ▷ Evaluation at  $y = 1$ 
4:  $A^{(2)} \leftarrow A(\infty) = A_1$  ▷ Evaluation at  $y = \infty$ 
5: return  $[A^{(0)}, A^{(1)}, A^{(2)}]$ 

```

Algorithm 18 Classical Karatsuba Interpolation**Input:** Products $[w_0, w_1, w_2]$ **Output:** Product polynomial $C(x)$

```

1:  $w_1 \leftarrow w_1 - w_0 - w_2$ 
2: return  $C(x) = w_2 y^2 + w_1 y + w_0$ 

```

Similarly, Algorithm 19 presents the conventional Toom-Cook 4-way polynomial multiplication algorithm, followed by its evaluation and interpolation routines in Algorithms 20 and 21, respectively. These algorithms constitute the classical counterparts of the proposed striding implementation and are used recursively across all security levels of ATLAS.

Together, these algorithms provide the classical foundation for the proposed striding polynomial multiplication framework, whose evaluation, interpolation, and hybrid decomposition strategies are described in the main text.

Algorithm 19 Toom-Cook 4-way Algorithm**Input:** Polynomials $A(x), B(x)$ of degree n **Output:** $C(x) = A(x) * B(x)$

```

1:  $A(y) = A_3 \cdot y^3 + A_2 \cdot y^2 + A_1 \cdot y + A_0, \quad y = x^{n/4}$     ▷ Split  $A(x)$  into four polynomials of size  $n/4$ 
2:  $B(y) = B_3 \cdot y^3 + B_2 \cdot y^2 + B_1 \cdot y + B_0$                 ▷ Split  $B(x)$  into four polynomials of size  $n/4$ 
   // Evaluation at  $y = \{0, \pm 1, \pm \frac{1}{2}, 2, \infty\}$ 
3:  $w_1 = A(\infty) * B(\infty) = A_3 * B_3$ 
4:  $w_2 = A(2) * B(2) = (A_0 + 2 \cdot A_1 + 4 \cdot A_2 + 8 \cdot A_3) * (B_0 + 2 \cdot B_1 + 4 \cdot B_2 + 8 \cdot B_3)$ 
5:  $w_3 = A(1) * B(1) = (A_0 + A_1 + A_2 + A_3) * (B_0 + B_1 + B_2 + B_3)$ 
6:  $w_4 = A(-1) * B(-1) = (A_0 - A_1 + A_2 - A_3) * (B_0 - B_1 + B_2 - B_3)$ 
7:  $w_5 = A(\frac{1}{2}) * B(\frac{1}{2}) = (8 \cdot A_0 + 4 \cdot A_1 + 2 \cdot A_2 + A_3) * (8 \cdot B_0 + 4 \cdot B_1 + 2 \cdot B_2 + B_3)$ 
8:  $w_6 = A(-\frac{1}{2}) * B(-\frac{1}{2}) = (8 \cdot A_0 - 4 \cdot A_1 + 2 \cdot A_2 - A_3) * (8 \cdot B_0 - 4 \cdot B_1 + 2 \cdot B_2 - B_3)$ 
9:  $w_7 = A(0) * B(0) = A_0 * B_0$ 
   // Interpolation
10:  $w_2 = w_2 + w_5, \quad w_6 = w_6 - w_5$ 
11:  $w_4 = (w_4 - w_3)/2, \quad w_2 = w_5 - w_1 - 64 \cdot w_7$ 
12:  $w_3 = w_3 + w_4, \quad w_5 = 2 \cdot w_5 - w_6$ 
13:  $w_2 = w_2 - 65 \cdot w_3, \quad w_3 = w_3 - w_7 - w_1$ 
14:  $w_2 = w_2 + 45 \cdot w_3, \quad w_5 = (w_5 - 8w_3)/24$ 
15:  $w_6 = w_6 + w_2, \quad w_2 = (w_2 + 16 \cdot w_4)/18$ 
16:  $w_4 = -(w_4 + w_2), \quad w_6 = (30 \cdot w_2 - w_6)/60$ 
17:  $w_2 = w_2 - w_6$ 
18: return  $C(x) = w_1 \cdot y^6 + w_2 \cdot y^5 + w_3 \cdot y^4 + w_4 \cdot y^3 + w_5 \cdot y^2 + w_6 \cdot y + w_7$ 

```

Algorithm 20 Classical Toom-4 Evaluation**Input:** Polynomial $A(x)$ of degree n **Output:** Evaluations $[A^{(0)}, A^{(1)}, \dots, A^{(6)}]$

```

1:  $A(y) = A_3 \cdot y^3 + A_2 \cdot y^2 + A_1 \cdot y + A_0, \quad y = x^{n/4}$     ▷ Split  $A(x)$  into four polynomials of size  $n/4$ 
2:  $A^{(0)} \leftarrow A(\infty) = A_3$                                 ▷ Evaluate at  $y = \infty$ 
3:  $A^{(1)} \leftarrow A(2) = A_0 + 2A_1 + 4A_2 + 8A_3$             ▷ Evaluate at  $y = 2$ 
4:  $A^{(2)} \leftarrow A(1) = A_0 + A_1 + A_2 + A_3$               ▷ Evaluate at  $y = 1$ 
5:  $A^{(3)} \leftarrow A(-1) = A_0 - A_1 + A_2 - A_3$              ▷ Evaluate at  $y = -1$ 
6:  $A^{(4)} \leftarrow A(\frac{1}{2}) = 8A_0 + 4A_1 + 2A_2 + A_3$         ▷ Evaluate at  $y = 1/2$ 
7:  $A^{(5)} \leftarrow A(-\frac{1}{2}) = 8A_0 - 4A_1 + 2A_2 - A_3$       ▷ Evaluate at  $y = -1/2$ 
8:  $A^{(6)} \leftarrow A(0) = A_0$                                 ▷ Evaluate at  $y = 0$ 
9: return  $[A^{(0)}, A^{(1)}, A^{(2)}, A^{(3)}, A^{(4)}, A^{(5)}, A^{(6)}]$ 

```

Algorithm 21 Classical Toom-4 Interpolation**Input:** Products $[w_1, w_2, \dots, w_7]$ **Output:** Product polynomial $C(x)$

```

1:  $w_2 \leftarrow w_2 + w_5$ 
2:  $w_6 \leftarrow w_6 - w_5$ 
3:  $w_4 \leftarrow (w_4 - w_3)/2$ 
4:  $w_2 \leftarrow w_5 - w_1 - 64 \cdot w_7$ 
5:  $w_3 \leftarrow w_3 + w_4$ 
6:  $w_5 \leftarrow 2 \cdot w_5 - w_6$ 
7:  $w_2 \leftarrow w_2 - 65 \cdot w_3$ 
8:  $w_3 \leftarrow w_3 - w_7 - w_1$ 
9:  $w_2 \leftarrow w_2 + 45 \cdot w_3$ 
10:  $w_5 \leftarrow (w_5 - 8 \cdot w_3)/24$ 
11:  $w_6 \leftarrow w_6 + w_2$ 
12:  $w_2 \leftarrow (w_2 + 16 \cdot w_4)/18$ 
13:  $w_4 \leftarrow -(w_4 + w_2)$ 
14:  $w_6 \leftarrow (30 \cdot w_1 - w_2 - w_6)/60$ 
15:  $w_2 \leftarrow w_2 - w_6$ 
16: return  $C(x) = w_1y^6 + w_2y^5 + w_3y^4 + w_4y^3 + w_5y^2 + w_6y + w_7$ 

```

C SPECIFICATION SUBROUTINES

This appendix lists the main subroutines appearing in the specification in the same order as they are introduced in the document.

1. **decompose_keygen**: Decomposes a coefficient into high-order and low-order parts during public-key generation.
2. **decompose_sign**: Decomposes a coefficient in the signing domain for hint generation and correctness control.
3. **make_hint**: Produces a one-bit hint indicating whether the high-order decomposition changes after perturbation.
4. **use_hint**: Uses the transmitted hint to recover the correct high-order component during verification.
5. **sig_keygen**: Generates the public key and secret key.
6. **sig_sign**: Produces a signature $\sigma = (\mathbf{z}, \mathbf{h}, c)$ on a message.
7. **sig_verify**: Verifies the signature using the public key and the message.
8. **pack_pk** / **unpack_pk**: Serialize and deserialize the public key.
9. **pack_sk** / **unpack_sk**: Serialize and deserialize the secret key.

10. `pack_sig` / `unpack_sig`: Serialize and deserialize the signature.
11. `pseudoXOF`: Expands seeds and digests into pseudorandom output for sampling and challenge derivation.
12. `expand_mat`: Deterministically generates the public matrix from the seed ρ .
13. `challenge`: Samples the sparse challenge polynomial from μ and $\mathbf{w}_1$.
14. `poly_uniform_eta`: Samples secret coefficients from the bounded distribution over $[-\eta, \eta]$.
15. `poly_uniform_gamma1m1`: Samples masking coefficients from the bounded distribution over $[-(\gamma_1 - 1), \gamma_1 - 1]$.

D AUTHOR DETAILS

The author names below are listed in alphabetical order.

SUPRIYA ADHIKARY

- *Affiliation*: IIT Kanpur
- *Phone Number*: +91 8348375299
- *E-mail*: `adhikarys@cse.iitk.ac.in`
- *Postal Address*: RM-511 (C), Rajeev Motwani Building, Department of Computer Science and Engineering, IIT Kanpur, Kalyanpur, India, Uttar Pradesh, 208016

DEBAYAN DAS

- *Affiliation*: IISc, Bangalore
- *Phone Number*: +91 8274039496
- *E-mail*: `debayandas@iisc.ac.in`
- *Postal Address*: Room 119, Department of Electronic Systems Engineering, Indian Institute of Science, Bangalore, India 560012

ABHINAVA DE

- *Affiliation*: Indian Institute of Science Bangalore
- *Phone Number*: (+91) 7003150983
- *E-mail*: `abhinavade@iisc.ac.in`
- *Postal Address*: 135, Department of Electronic Systems Engineering, Indian Institute of Science, Bangalore, INDIA 560012

ANINDYA GANGULY

- *Affiliation:* Indian Institute of Technology Kanpur
- *Phone Number:* (+91) 7872648932
- *E-mail:* anindyag@cse.iitk.ac.in
- *Postal Address:* KD-213; Kadim Diwan Building, Department of Computer Science and Engineering, Indian Institute of Technology Kanpur, Kalyanpur, Kanpur, Uttar Pradesh, INDIA, 208016

HARRY HART

- *Affiliation:* University of Surrey
- *Phone Number:* +447925797137
- *E-mail:* h.hart@surrey.ac.uk
- *Postal Address:* Computer Science Research Centre, University of Surrey, Guildford, Surrey, GU2 7XH, United Kingdom

ANGSHUMAN KARMAKAR

- *Affiliation:* IIT Kanpur & Qryptshield Technologies Private Limited
- *Phone Number:* +91 8967827714
- *E-mail:* angshuman@cse.iitk.ac.in
- *Postal Address:* 406, Rajeev Motwani Building. Department of Computer Science and Engineering, IIT Kanpur, Kalyanpur, India, Uttar Pradesh, 208016

SUPARNA KUNDU

- *Affiliation:* COSIC, KU Leuven
- *Phone Number:* +32467639966
- *E-mail:* Suparna.Kundu@esat.kuleuven.be
- *Postal Address:* Kasteelpark Arenberg 10 - box 2452, 3001 Leuven, Belgium

CHAOYUN LI

- *Affiliation:* University of Surrey
- *Phone Number:* +44 (0)1483 686058
- *E-mail:* `c.li@surrey.ac.uk`
- *Postal Address:* Computer Science Research Centre, University of Surrey, Guildford, Surrey, GU2 7XH, United Kingdom

PRAJNA MAHADEV

- *Affiliation:* IISc, Bangalore
- *Phone Number:* (+91) 8123181898
- *E-mail:* `prajnam24@gmail.com`
- *Postal Address:* Department of Electronic Systems Engineering, Indian Institute of Science, Bangalore, India 560012

PUJA MONDAL

- *Affiliation:* Indian Institute of Technology Kanpur
- *Phone Number:* (+91) 8240254291
- *E-mail:* `pujamondal@cse.iitk.ac.in`
- *Postal Address:* RM-511; Rajeev Motwani Building, Department of Computer Science and Engineering, Indian Institute of Technology Kanpur, Kalyanpur, Kanpur, Uttar Pradesh, INDIA, 208016

QUINTEN NORGA

- *Affiliation:* COSIC, KU Leuven
- *Phone Number:* +32 16 32 10 50
- *E-mail:* `quinten.norga@esat.kuleuven.be`
- *Postal Address:* Kasteelpark Arenberg 10, bus 2452, B-3001 Leuven-Heverlee, Belgium

DEBRANJAN PAL

- *Affiliation:* The LNM Institute of Information Technology
- *Phone Number:* (+91) 9830749208
- *E-mail:* `debranjan.crl@gmail.com`
- *Postal Address:* The LNM Institute of Information Technology, Rupa ki Nangal, Post-Sumel, Via-Jamdoli, Jaipur, Rajasthan, 302031

KARTHICK SRIVATSAN

- *Affiliation:* IIT Kanpur
- *Phone Number:* +91 8148860636
- *E-mail:* `karthicks24@iitk.ac.in`
- *Postal Address:* RM-511; Rajeev Motwani Building, Department of Computer Science and Engineering, Indian Institute of Technology Kanpur, Kalyanpur, Kanpur, Uttar Pradesh, INDIA, 208016.